Федеральное государственное бюджетное образовательное учреждение
высшего образования
«Чувашский государственный университет им. И.Н. Ульянова»
Кафедра вычислительной техники

# Лабораторная работа № 6
# Индикация параметров модели ПО «Модель ОС»
По дисциплине “Операционные системы”

Выполнил: студент группы ИВТ-42-23
Иванов И.Л.

Проверил: кандидат технических наук
Ковалев С.В.

Чебоксары, 2026

Цель: Разработать интерфейс модели индикации параметров модели ПО «Модель ОС».

Исходники: https://github.com/kukirmash/os_model/

**Задание**

1. Разделить процедуру индикации на подпрограммы:

- «Вывод постоянной информации» (заголовки, язык управления, номера процессов, названия параметров);

- «Вывод параметров системы»;

- «Вывод параметров нового задания»;

- «Вывод параметров центральных процессоров»;

- «Вывод параметров процессов» (всех параметров всех процессов на одном окне);

- «Вывод последовательности выполнения процессов на процессоре» (максимум 16 номеров последних процессов);

- «Вывод средних значений параметров процессов».

2. Модель должна выводить данные, позволяющие анализировать эффективность используемого метода распределения времени процессоров (время ожидания, время выполнения процесса, длины очередей, производительность системы, полезная загрузка процессора и т. д.) в зависимости от параметров задания и системы.

3. Вся информация должна отображаться сразу на одном экране и быть стабильной.

4. Предусмотреть вывод средних значений входных и выходных параметров процессов.

Если требуется, добавить необходимые переменные.

Получить результаты: индикацию параметров модели в соответствии с индивидуальным заданием (вариант 11 — относительные и динамические приоритеты).

В отчёте представить: блок-схемы алгоритмов, анализ программного кода, листинг программы, экранные формы.

В текущей версии модели основной пользовательский интерфейс — графический (Wails), поэтому индикация реализована в виде разделов единого экрана; разделение процедуры индикации на подпрограммы выполнено как на уровне ядра (сбор статистики и формирование снимка состояния), так и на уровне функций отрисовки интерфейса.

**БСА**

Блок-схемы алгоритмов, реализованных в данной работе, представлены на рисунках 1–5. При построении использованы символы по ГОСТ 19.701-90: терминатор — скруглённый прямоугольник, процесс — прямоугольник, предопределённый процесс (вызов подпрограммы) — прямоугольник с двойными боковыми линиями, решение — ромб, подготовка к циклу — шестиугольник, вывод данных — параллелограмм.

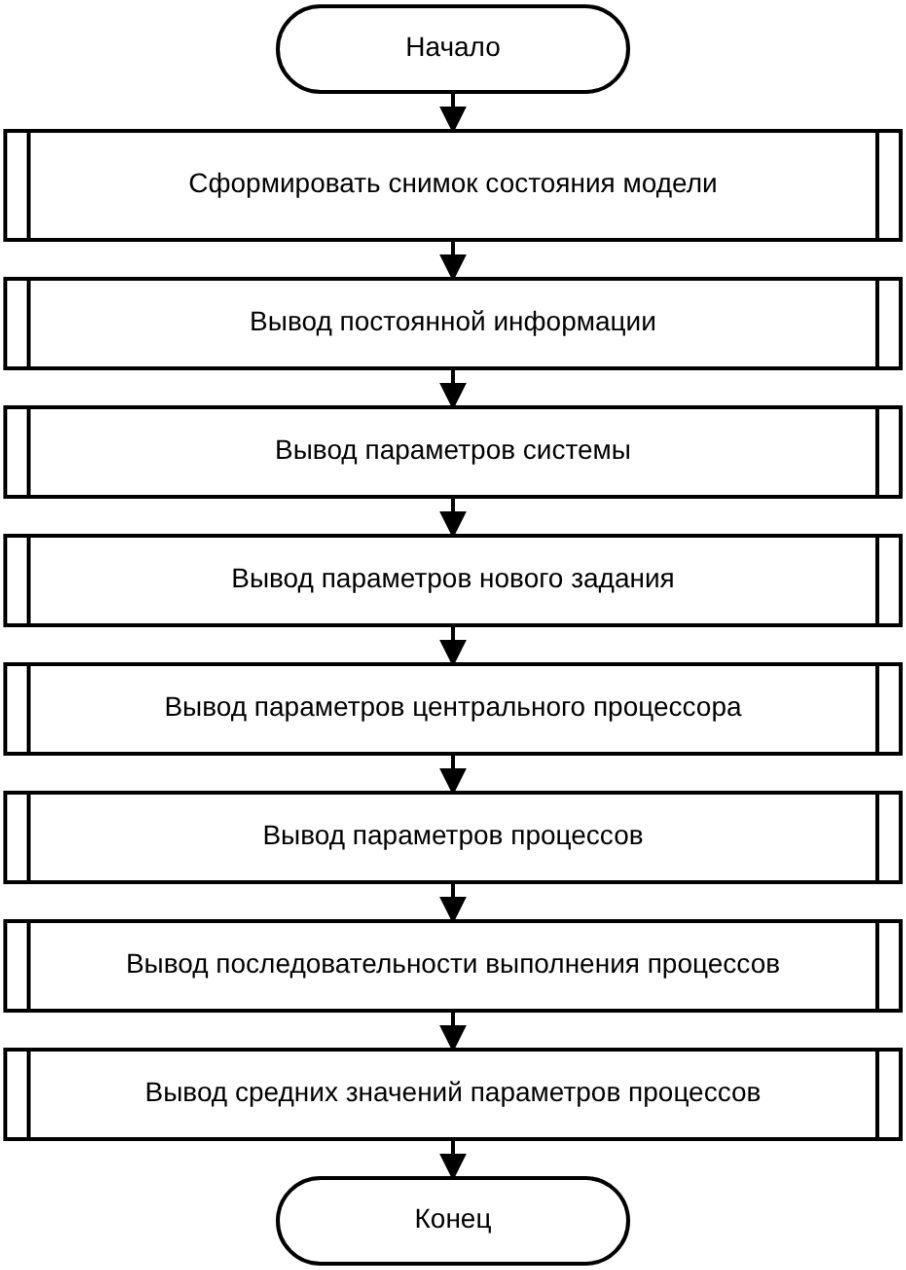



Рисунок 1 — Структура подпрограммы индикации (А)

Подпрограмма индикации (рисунок 1) разделена на семь подпрограмм в соответствии с заданием. Сначала формируется снимок состояния модели, затем на одном экране выводятся: постоянная информация (заголовки, язык управления, названия параметров), параметры системы, параметры нового задания, параметры центрального процессора, параметры процессов, последовательность выполнения процессов на процессоре и средние значения параметров процессов.

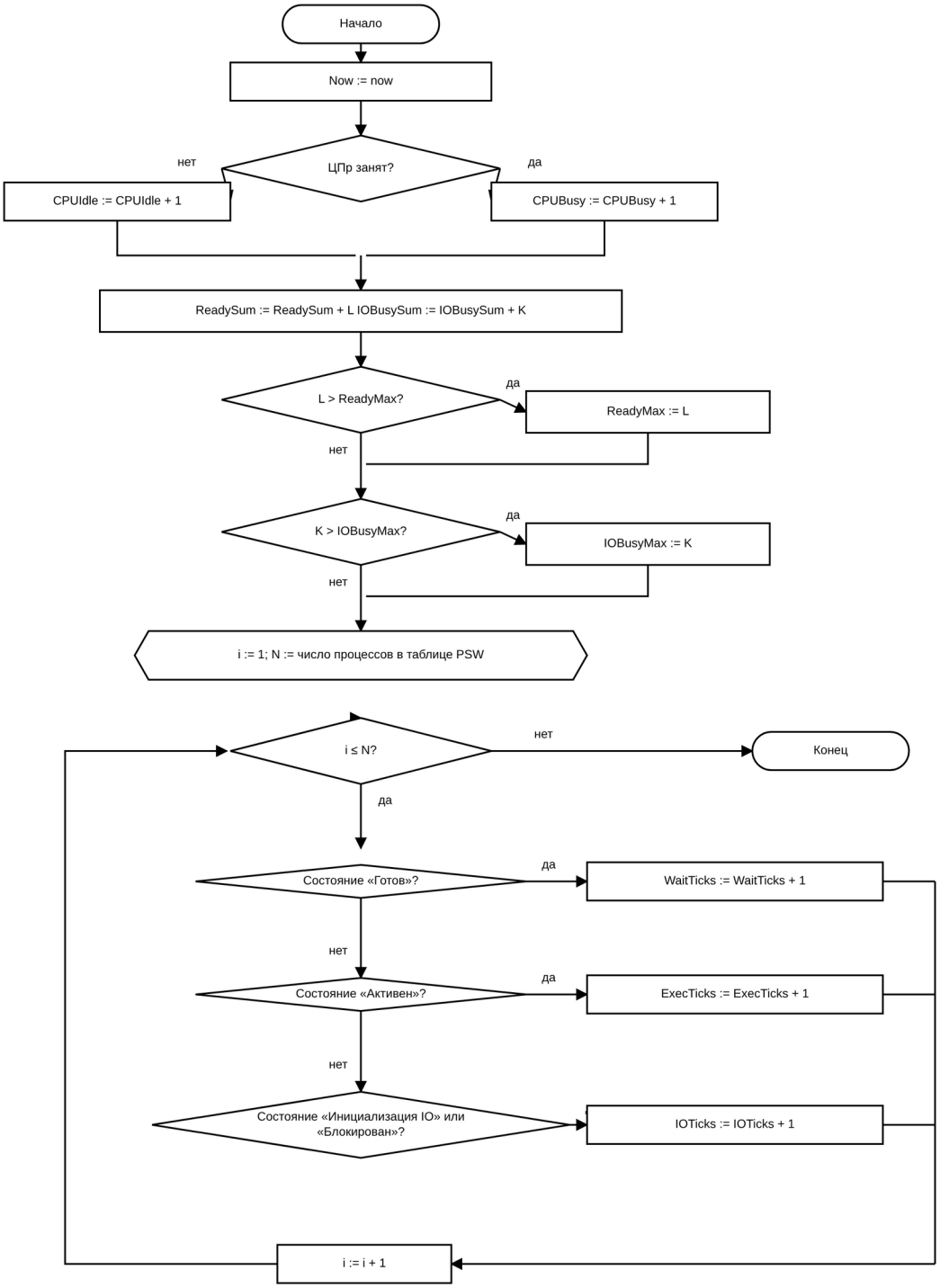



Рисунок 2 — Блок-схема подпрограммы сбора статистики за такт (Б)

Подпрограмма сбора статистики (рисунок 2) выполняется каждый такт модели. Фиксируются занятость центрального процессора, длины очередей (сумма для средней и максимум) и времена каждого процесса по его текущему состоянию: «Готов» — ожидание, «Активен» — выполнение на ЦПр, «Инициализация IO» и «Блокирован» — ввод-вывод. Цикл по таблице слов состояний процессов выделен штриховой петлёй возврата.

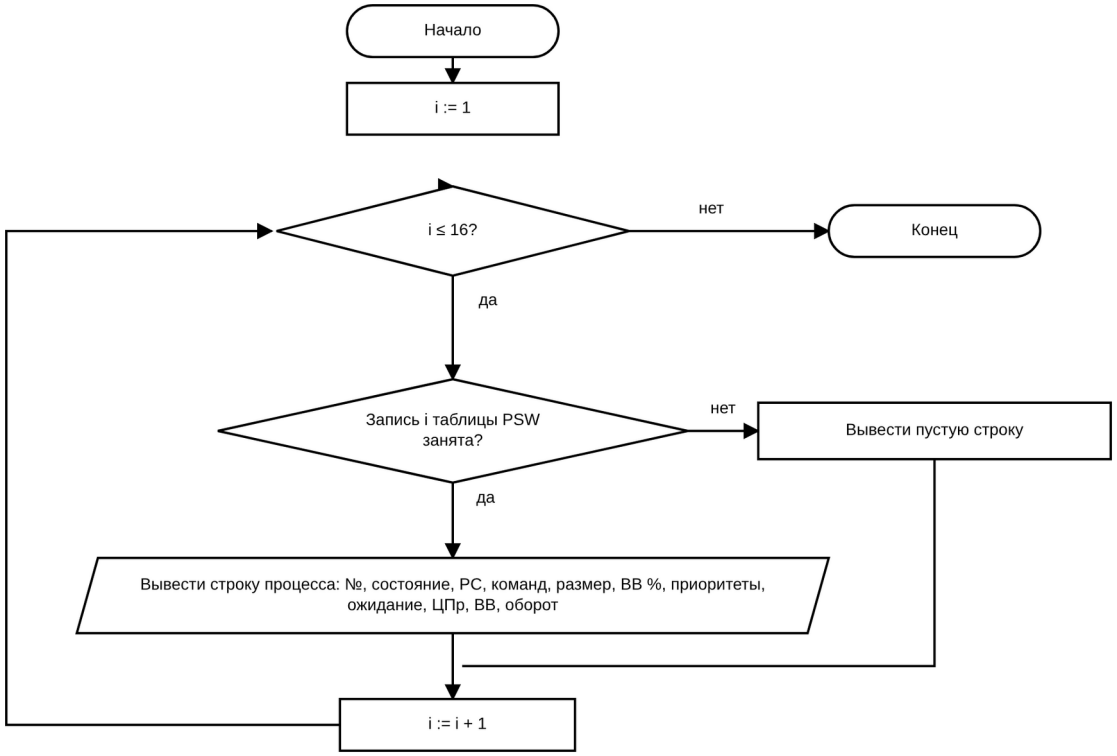



Рисунок 3 — Блок-схема подпрограммы «Вывод параметров процессов» (В)

Подпрограмма вывода параметров процессов (рисунок 3) выводит все параметры всех процессов на одном окне: проходится весь диапазон записей таблицы слов состояний процессов (до 16), занятые записи выводятся со всеми параметрами, свободные — пропускаются.

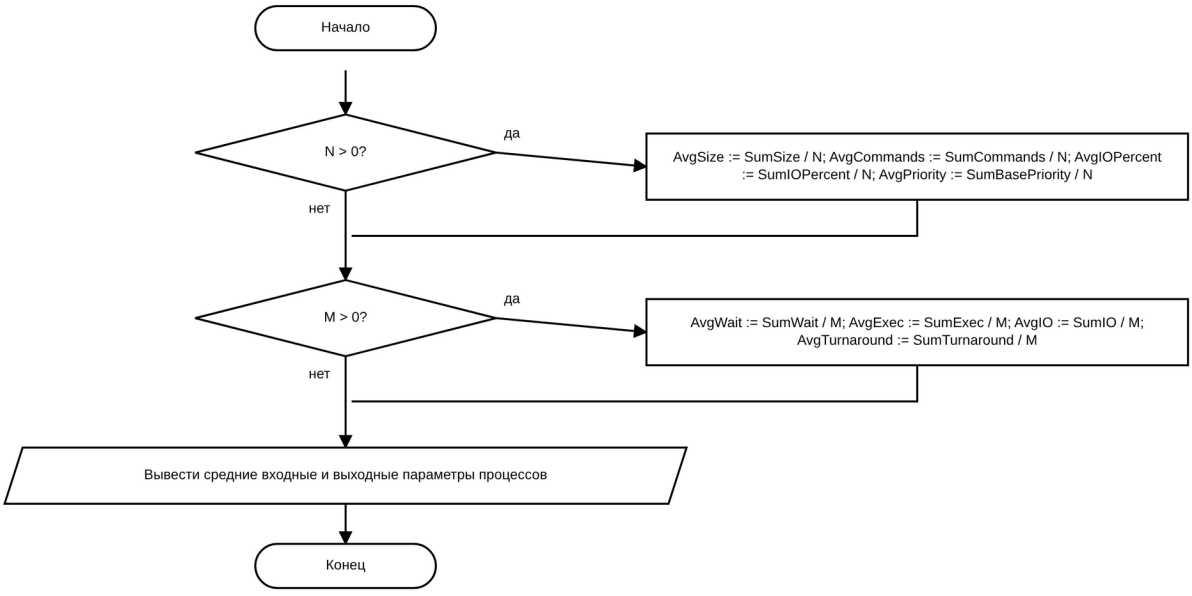



Рисунок 4 — Блок-схема подпрограммы «Вывод средних значений параметров процессов» (Г)

Подпрограмма вывода средних значений (рисунок 4) вычисляет средние входные параметры по всем загруженным заданиям и средние выходные параметры по завершённым процессам. Деление выполняется только при ненулевом числе слагаемых, иначе средние принимаются равными нулю.

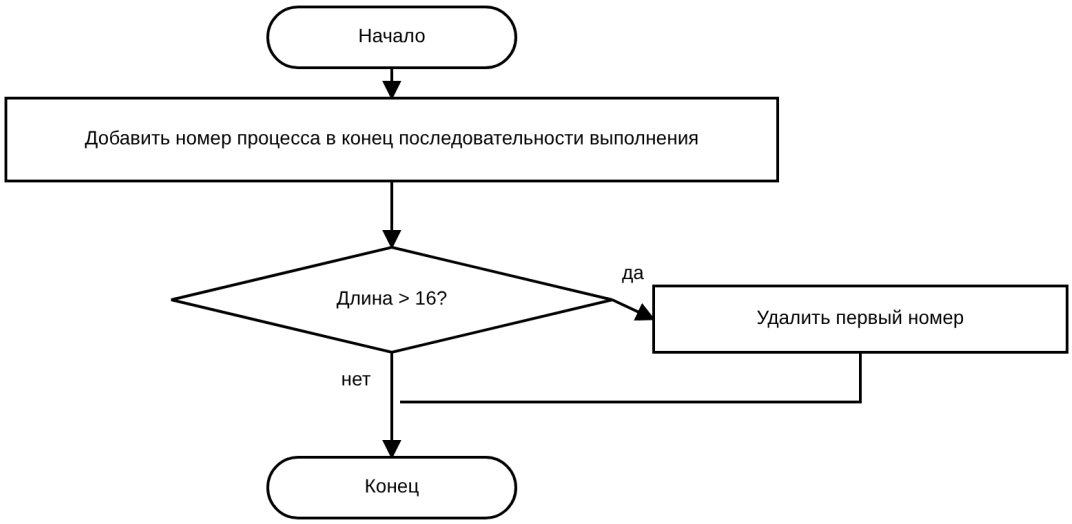



Рисунок 5 — Блок-схема подпрограммы ведения последовательности выполнения процессов (Д)

Подпрограмма ведения последовательности выполнения (рисунок 5) вызывается при каждом выборе процесса планировщиком: номер процесса добавляется в конец последовательности, а при переполнении удаляется самый старый номер — хранятся 16 последних процессов, выполнявшихся на центральном процессоре.

## Анализ программного кода

Индикация построена по трёхуровневой схеме. Ядро модели ведёт учёт статистики: времена процессов хранятся в слове состояния процесса, суммарные показатели — в новом модуле internal/stats. Снимок состояния (Snapshot) собирает все данные в единую структуру. Интерфейс (frontend/dist) отображает снимок на одном экране отдельными подпрограммами. Такое разделение позволяет использовать статистику не только в интерфейсе, но и в автоматических тестах.

**1. Слово состояния процесса (internal/process/psw.go)**

В PSW добавлены переменные, необходимые для учёта времён процесса: такт загрузки LoadedAt и счётчики тактов, проведённых процессом в различных состояниях. Время ожидания (WaitTicks) накапливается в состоянии «Готов», время выполнения на ЦПр (ExecTicks) — в состоянии «Активен», время ввода-вывода (IOTicks) — в состояниях «Инициализация IO» и «Блокирован». Оборотное время процесса вычисляется как разность текущего такта модели и такта загрузки (Time − LoadedAt).

```
// PSW (Process Status Word, слово состояния процесса) описывает
// динамический процесс, порождённый заданием.
//
// Все изменения состояния процесса выполняет только регулировщик
// (подпрограмма Process), остальные модули лишь читают эти поля.
// Ход операции ввода-вывода (номер процессора ВВ и счётчик тактов)
// хранится в состоянии процессора ВВ — см. пакет io.
type PSW struct {
    ID              int   // идентификатор процесса
    Task            *Task // ссылка на задание, породившее процесс
    PC              int   // счётчик команд
    State           State // текущее состояние
    DynamicPriority int   // текущий (динамический) приоритет


    // Статистика времени процесса (лабораторная работа 6): заполняется
    // ядром модели каждый такт по текущему состоянию процесса.
    LoadedAt  int // такт модели, в котором процесс загружен
    WaitTicks int // время ожидания в очереди готовности, тактов
    ExecTicks int // время выполнения на центральном процессоре, тактов
    IOTicks   int // время в состоянии «Инициализация IO» и «Блокирован», тактов
}
```

## 2. Модуль статистики (internal/stats/stats.go)

Модуль ведёт накопленные показатели системы: занятость центрального процессора (CPUBusy, CPUIdle), суммы длин очереди готовности и занятых процессоров ВВ (ReadySum, IOBusySum) с максимумами (ReadyMax, IOBusyMax), последовательность выполнения процессов (ExecHistory), накопители входных параметров загруженных заданий (SumSize, SumCommands, SumIOPercent, SumBasePriority) и выходных параметров завершённых процессов (SumWait, SumExec, SumIO, SumTurnaround).

Подпрограмма Collect вызывается ядром каждый такт и фиксирует состояние модели: занятость процессора, длины очередей и времена процессов по их состояниям.

```
// Collect фиксирует состояние модели за один такт: занятость процессора,
// длины очередей и времена процессов по их текущим состояниям.
// Вызывается ядром каждый такт после диспетчеризации и до выполнения
// команды. Состояния процессов не изменяются.
```

```
func (s *Stats) Collect(now int, cpuBusy bool, table []*process.PSW, readyLen, ioBusy
int) {
    s.Now = now


    // Занятость центрального процессора.
    if cpuBusy {
        s.CPUBusy++
    } else {
        s.CPUIdle++
    }


    // Длина очереди готовности: сумма для средней и максимум.
    s.ReadySum += readyLen
    if readyLen > s.ReadyMax {
        s.ReadyMax = readyLen
    }


    // Занятые процессоры ввода-вывода: сумма и максимум.
    s.IOBusySum += ioBusy
    if ioBusy > s.IOBusyMax {
        s.IOBusyMax = ioBusy
    }


    // Время процессов: каждый такт процесс проводит ровно в одном
    // состоянии — ожидания, выполнения или ввода-вывода.
    for _, p := range table {
        switch p.State {
        case process.StateReady:
            p.WaitTicks++
        case process.StateActive:
            p.ExecTicks++
        case process.StateInitIO, process.StateBlockedIO:
            p.IOTicks++
        }
    }
}
```

Подпрограмма OnLoad регистрирует загруженное задание (запоминает такт загрузки и накапливает входные параметры), подпрограмма OnComplete — накапливает выходные параметры завершённого процесса, подпрограмма OnDispatch — ведёт последовательность выполнения процессов на ЦПр.

```
// OnLoad регистрирует загруженное задание: запоминает такт загрузки
// и накапливает входные параметры для средних. Вызывается регулировщиком.
func (s *Stats) OnLoad(p *process.PSW) {
    p.LoadedAt = s.Now


    s.LoadedCount++
    s.SumSize += p.Task.Size
    s.SumCommands += p.Task.TotalCommands
    s.SumIOPercent += p.Task.IOPercent
    s.SumBasePriority += p.Task.BasePriority
}
// OnComplete накапливает выходные параметры завершённого процесса:
// времена ожидания, выполнения, ввода-вывода и оборотное время.
// Вызывается регулировщиком при выгрузке процесса.
func (s *Stats) OnComplete(p *process.PSW) {
    s.CompletedCount++
    s.SumWait += p.WaitTicks
    s.SumExec += p.ExecTicks
    s.SumIO += p.IOTicks
    s.SumTurnaround += s.Now - p.LoadedAt
}
// OnDispatch запоминает номер процесса, выбранного планировщиком для
// выполнения на центральном процессоре. Хранятся последние HistoryLimit
```

```
// номеров — «последовательность выполнения процессов на процессоре».
func (s *Stats) OnDispatch(id int) {
    s.ExecHistory = append(s.ExecHistory, id)
    if len(s.ExecHistory) > HistoryLimit {
        s.ExecHistory = s.ExecHistory[len(s.ExecHistory)-HistoryLimit:]
    }
}
```

Средние значения и производные показатели вычисляются подпрограммами модуля:

$$AvgWait = SumWait / M, \quad AvgExec = SumExec / M,$$

$$AvgIO = SumIO / M, \quad AvgTurnaround = SumTurnaround / M,$$

где M — число завершённых процессов;

$$AvgSize = SumSize / N, \quad AvgCommands = SumCommands / N,$$

$$AvgIOPercent = SumIOPercent / N, \quad AvgPriority = SumBasePriority / N,$$

где N — число загруженных заданий;

$$P_load = CPUBusy / Now \cdot 100\ \% \quad \text{(полезная загрузка ЦПр)},$$

$$P_sys = Completed \cdot 1000 / Now \quad \text{(производительность, заданий на 1000 тактов)},$$

$$L_sr = ReadySum / Now, \quad L_max = ReadyMax \quad \text{(длина очереди готовности)}.$$

**3. Регулировщик (internal/regulator/regulator.go)**

Начало и окончание «жизни» процесса фиксируются в регулировщике — единственном месте, где изменяются состояния процессов. При загрузке задания вызывается подпрограмма OnLoad, при выгрузке — OnComplete; тем самым статистика не может «потерять» процесс.

```
p.State = process.StateReady
r.Table = append(r.Table, p)
r.Planner.Add(p)
if r.Stats != nil {
    r.Stats.OnLoad(p)
}
p.State = process.StateNone
r.Completed++
if r.Stats != nil {
    r.Stats.OnComplete(p)
}
```

**4. Ядро модели (internal/os/system.go)**

В алгоритм одного такта добавлен шаг сбора статистики (Сб), выполняемый после шага А (диспетчеризации) и до шага Б1. Состояние фиксируется на начало выполнения такта: каждый процесс в этот момент находится ровно в одном состоянии, поэтому сумма его времён равна

времени пребывания в системе, а суммарное время выполнения на ЦПр совпадает с занятостью процессора.

```
func (sys *System) DoOneTick() {
    sys.Time++


    // А. Инициализация цикла.
    if sys.CPU.State == cpu.StateIdle {
        sys.dispatchProcess()
    }


    // Сб. Учёт статистики за текущий такт: состояние фиксируется после
    // диспетчеризации и до выполнения команды.
    sys.collectStats()


    // Б1. Моделирование центрального процессора.
    sys.CPU.Tick()


    // Б2. Моделирование процессоров ввода-вывода и обработка прерываний.
    sys.IO.TickIO(sys.Time)
    sys.HandleInterrupts()


    // Б3. Прерывания по времени не моделируются (см. комментарий выше).


    // Б4. Проверка достигнутого состояния системы.
    sys.CheckSystemState()


    // Б5. Обновление баз данных модели ОС.
    sys.Regulator.Planner.UpdatePriorities()


    // Б6. Отображение изменений в состоянии системы.
    sys.notify()
}
```

Подпрограмма collectStats передаёт модулю статистики текущие данные ядра, а подпрограмма dispatchProcess регистрирует выбор процесса планировщиком — номер процесса попадает в последовательность выполнения.

```
// collectStats — подпрограмма сбора статистики за такт (лабораторная
// работа 6): занятость центрального процессора, длины очередей и времена
// процессов по их состояниям.
func (sys *System) collectStats() {
    sys.Stats.Collect(
        sys.Time,
        sys.CPU.State == cpu.StateWorking,
        sys.Regulator.Table,
        len(sys.Regulator.Planner.ReadyQueue),
        len(sys.IO.Operations),
    )
}
```

Параметры последнего сгенерированного задания сохраняются в поле LastTask (тип LastTaskInfo): задание может быть не загружено, если нет свободной записи в таблице PSW или недостаточно памяти, — интерфейс отображает и параметры заявки, и причину отказа.

```
// LastTaskInfo — параметры последнего сгенерированного задания: выводятся
```

```
// подпрограммой индикации «Вывод параметров нового задания» (лабораторная
// работа 6). Задание может быть не загружено, если не хватило ресурсов.
type LastTaskInfo struct {
    Task   process.Task // параметры задания
    Loaded bool         // задание загружено в память
    Reason string       // причина, по которой задание не загружено
}
```

## 5. Снимок состояния (internal/os/snapshot.go)

Снимок состояния — единый источник данных для индикации. Структура ProcessInfo дополнена временами процесса (ожидание, выполнение, ввод-вывод, оборотное время), добавлены TaskInfo (параметры нового задания) и StatsInfo (статистика модели). Оборотное время вычисляется при формировании снимка по текущему такту модели.

```
// StatsInfo — статистика модели для индикации (лабораторная работа 6).
type StatsInfo struct {
    // Центральный процессор.
    CPUBusy        int     `json:"cpuBusy"`        // тактов процессор выполнял
команды
    CPUIdle        int     `json:"cpuIdle"`        // тактов процессор простаивал
    CPUUtilization float64 `json:"cpuUtilization"` // полезная загрузка ЦПр, %


    // Длины очередей.
    ReadyLen  int     `json:"readyLen"`  // текущая длина очереди готовности
    ReadyAvg  float64 `json:"readyAvg"`  // средняя длина очереди готовности
    ReadyMax  int     `json:"readyMax"`  // максимальная длина очереди
    IOBusy    int     `json:"ioBusy"`    // занятых процессоров ВВ сейчас
    IOBusyAvg float64 `json:"ioBusyAvg"` // среднее число занятых процессоров ВВ
    IOBusyMax int     `json:"ioBusyMax"` // максимум занятых процессоров ВВ


    // Производительность системы: выполненных заданий на 1000 тактов.
    Throughput float64 `json:"throughput"`


    // Последовательность выполнения процессов на ЦПр (до 16 номеров).
    ExecHistory []int `json:"execHistory"`


    // Средние входные параметры загруженных заданий.
    LoadedCount  int     `json:"loadedCount"`
    AvgSize      float64 `json:"avgSize"`
    AvgCommands  float64 `json:"avgCommands"`
    AvgIOPercent float64 `json:"avgIoPercent"`
    AvgPriority  float64 `json:"avgPriority"`


    // Средние выходные параметры завершённых процессов.
    CompletedCount int     `json:"completedCount"`
    AvgWait        float64 `json:"avgWait"`       // ожидание, тактов
    AvgExec        float64 `json:"avgExec"`       // выполнение на ЦПр, тактов
    AvgIO          float64 `json:"avgIo"`         // ввод-вывод, тактов
    AvgTurnaround  float64 `json:"avgTurnaround"` // оборотное время, тактов
}
// processInfos преобразует список PSW в данные для индикации.
// now — текущий такт модели: по нему вычисляется оборотное время.
func processInfos(psws []*process.PSW, now int) []ProcessInfo {
    infos := make([]ProcessInfo, 0, len(psws))
    for _, p := range psws {
        infos = append(infos, ProcessInfo{
            ID:              p.ID,
            Size:            p.Task.Size,
            PC:              p.PC,
            TotalCommands:   p.Task.TotalCommands,
```

```
            IOPercent:       p.Task.IOPercent,
            State:           int(p.State),
            DynamicPriority: p.DynamicPriority,
            BasePriority:    p.Task.BasePriority,
            WaitTicks:       p.WaitTicks,
            ExecTicks:       p.ExecTicks,
            IOTicks:         p.IOTicks,
            Turnaround:      now - p.LoadedAt,
        })
    }
    return infos
}
```

## 6. Интерфейс модели (frontend/dist)

Каждой подпрограмме индикации по заданию соответствует отдельный раздел единого экрана:

| **Подпрограмма по заданию** | **Реализация** | **Раздел интерфейса** |
|---|---|---|
| Вывод постоянной информации | index.html, app.js | Заголовок, язык управления, названия параметров |
| Вывод параметров системы | renderSystem(), Snapshot.Stats | «Параметры системы» |
| Вывод параметров нового задания | renderTask(), Snapshot.LastTask | «Параметры нового задания» |
| Вывод параметров центральных процессоров | renderCPU(), Snapshot.Stats | «Параметры центрального процессора» |
| Вывод параметров процессов | renderProcesses(), Snapshot.Processes | «Параметры процессов (таблица PSW)» |
| Вывод последовательности выполнения процессов | renderHistory(), Stats.ExecHistory | «Последовательность выполнения процессов на процессоре» |
| Вывод средних значений параметров процессов | renderAverages(), методы Stats | «Средние значения параметров процессов» |

Постоянная информация (заголовки, язык управления, названия параметров) задана статически в файле index.html. Динамические данные подставляет подпрограмма render(snapshot) в файле app.js: она вызывает отдельные подпрограммы отрисовки панелей.

```
function render(snapshot) {
  renderHeader(snapshot);
  renderSystem(snapshot);
  renderCPU(snapshot);
  renderTask(snapshot);
  renderAverages(snapshot);
  renderHistory(snapshot);
  renderReady(snapshot);
  renderIO(snapshot);
  renderInterrupts(snapshot);
  renderProcesses(snapshot);
}
```

Период отправки снимков в интерфейс ограничен 50 мс (константа uiRefreshInterval в app.go), поэтому при скорости до 1000 тактов в секунду интерфейс не перегружается, а отображение остаётся стабильным. Разделы имеют фиксированные позиции в сетке, поэтому элементы экрана не

смещаются при изменении числа процессов. Оформление вынесено в style.css.

### 7. Автоматическая проверка (internal/os/system_test.go)

Тест прогоняет модель несколько тысяч тактов и дополнительно проверяет согласованность статистики: число занятых и простаивающих тактов равно модельному времени; сумма времён каждого процесса по состояниям равна времени его пребывания в системе; суммарное время выполнения процессов на ЦПр совпадает с числом занятых тактов процессора; счётчики загруженных и завершённых процессов в статистике совпадают с данными ядра; длина последовательности выполнения не превышает 16 номеров. Тест подтверждает корректность учёта времён, очередей и средних значений.

## Листинг программы

### Файл internal/process/psw.go:

*Слово состояния процесса со счётчиками времени.*

```
package process

// State — состояние процесса в модели.
//
// Список возможных состояний по лабораторной работе 5:
// «Готов», «Активен», «Инициализация IO», «Блокирован».
// StateNone используется при выгрузке процесса (запись в таблице удаляется).
type State int

//
---------------------------------------------------------------------------
--
const (
    StateNone      State = iota // процесс не порождён / выгружен
    StateReady                  // «Готов»: находится в очереди планировщика
    StateActive                 // «Активен»: выполняется на центральном процессоре
    StateInitIO                 // «Инициализация IO»: инициализируется операция
ввода-вывода
    StateBlockedIO              // «Блокирован»: ожидает окончания операции ввода-
вывода
)

//
---------------------------------------------------------------------------
--
// PSW (Process Status Word, слово состояния процесса) описывает
// динамический процесс, порождённый заданием.
//
// Все изменения состояния процесса выполняет только регулировщик
// (подпрограмма Process), остальные модули лишь читают эти поля.
// Ход операции ввода-вывода (номер процессора ВВ и счётчик тактов)
// хранится в состоянии процессора ВВ — см. пакет io.
type PSW struct {
```

```
    ID              int   // идентификатор процесса
    Task            *Task // ссылка на задание, породившее процесс
    PC              int   // счётчик команд
    State           State // текущее состояние
    DynamicPriority int   // текущий (динамический) приоритет


    // Статистика времени процесса (лабораторная работа 6): заполняется
    // ядром модели каждый такт по текущему состоянию процесса.
    LoadedAt  int // такт модели, в котором процесс загружен
    WaitTicks int // время ожидания в очереди готовности, тактов
    ExecTicks int // время выполнения на центральном процессоре, тактов
    IOTicks   int // время в состоянии «Инициализация IO» и «Блокирован», тактов
}
```

## Файл internal/stats/stats.go:

*Модуль сбора статистики модели.*

```
// Package stats — модуль сбора статистики модели операционной системы.
//
// Статистика ведётся для анализа эффективности метода распределения времени
// центрального процессора (лабораторная работа 6): времена ожидания и
// выполнения процессов, длины очередей, полезная загрузка процессора,
// производительность системы и средние значения параметров процессов.
//
// Времена процесса (ожидание, выполнение, ввод-вывод) хранятся в слове
// состояния процесса — см. поля PSW в пакете process. Пакет stats хранит
// суммарные значения по системе и накопители для вычисления средних.
package stats


import "os_model/internal/process"


//
----------------------------------------------------------------------------------
--
// HistoryLimit — сколько последних процессов, выполнявшихся на процессоре,
// хранится для индикации (лабораторная работа 6).
const HistoryLimit = 16


//
----------------------------------------------------------------------------------
--
// Stats — накопленная статистика модели.
type Stats struct {
    Now int // текущий такт модели (обновляется подпрограммой Collect)


    // Центральный процессор.
    CPUBusy int // тактов процессор выполнял команды активного процесса
    CPUIdle int // тактов процессор простаивал


    // Очередь готовности.
    ReadySum int // сумма длин очереди готовности по тактам
    ReadyMax int // максимальная длина очереди за время моделирования


    // Процессоры ввода-вывода.
    IOBusySum int // сумма занятых процессоров ВВ по тактам
    IOBusyMax int // максимум одновременно занятых процессоров ВВ


    // Последовательность выполнения процессов на ЦПр (до HistoryLimit
    // последних номеров).
    ExecHistory []int


    // Входные параметры всех загруженных заданий (накопители средних).
    LoadedCount     int
    SumSize         int
```

```
    SumCommands     int
    SumIOPercent    int
    SumBasePriority int


    // Выходные параметры завершённых процессов (накопители средних).
    CompletedCount int
    SumWait        int
    SumExec        int
    SumIO          int
    SumTurnaround  int
}

//
-----------------------------------------------------------------------------
--
// OnLoad регистрирует загруженное задание: запоминает такт загрузки
// и накапливает входные параметры для средних. Вызывается регулировщиком.
func (s *Stats) OnLoad(p *process.PSW) {
    p.LoadedAt = s.Now


    s.LoadedCount++
    s.SumSize += p.Task.Size
    s.SumCommands += p.Task.TotalCommands
    s.SumIOPercent += p.Task.IOPercent
    s.SumBasePriority += p.Task.BasePriority
}

//
-----------------------------------------------------------------------------
--
// OnComplete накапливает выходные параметры завершённого процесса:
// времена ожидания, выполнения, ввода-вывода и оборотное время.
// Вызывается регулировщиком при выгрузке процесса.
func (s *Stats) OnComplete(p *process.PSW) {
    s.CompletedCount++
    s.SumWait += p.WaitTicks
    s.SumExec += p.ExecTicks
    s.SumIO += p.IOTicks
    s.SumTurnaround += s.Now - p.LoadedAt
}

//
-----------------------------------------------------------------------------
--
// OnDispatch запоминает номер процесса, выбранного планировщиком для
// выполнения на центральном процессоре. Хранятся последние HistoryLimit
// номеров — «последовательность выполнения процессов на процессоре».
func (s *Stats) OnDispatch(id int) {
    s.ExecHistory = append(s.ExecHistory, id)
    if len(s.ExecHistory) > HistoryLimit {
        s.ExecHistory = s.ExecHistory[len(s.ExecHistory)-HistoryLimit:]
    }
}

//
-----------------------------------------------------------------------------
--
// Collect фиксирует состояние модели за один такт: занятость процессора,
// длины очередей и времена процессов по их текущим состояниям.
// Вызывается ядром каждый такт после диспетчеризации и до выполнения
// команды. Состояния процессов не изменяются.
func (s *Stats) Collect(now int, cpuBusy bool, table []*process.PSW, readyLen, ioBusy
int) {
    s.Now = now


    // Занятость центрального процессора.
    if cpuBusy {
```

```
        s.CPUBusy++
    } else {
        s.CPUIdle++
    }

    // Длина очереди готовности: сумма для средней и максимум.
    s.ReadySum += readyLen
    if readyLen > s.ReadyMax {
        s.ReadyMax = readyLen
    }

    // Занятые процессоры ввода-вывода: сумма и максимум.
    s.IOBusySum += ioBusy
    if ioBusy > s.IOBusyMax {
        s.IOBusyMax = ioBusy
    }

    // Время процессов: каждый такт процесс проводит ровно в одном
    // состоянии — ожидания, выполнения или ввода-вывода.
    for _, p := range table {
        switch p.State {
        case process.StateReady:
            p.WaitTicks++
        case process.StateActive:
            p.ExecTicks++
        case process.StateInitIO, process.StateBlockedIO:
            p.IOTicks++
        }
    }
}

//
-----------------------------------------------------------------------------
--
// History возвращает копию последовательности выполнения процессов на ЦПр.
func (s *Stats) History() []int {
    history := make([]int, len(s.ExecHistory))
    copy(history, s.ExecHistory)
    return history
}

//
-----------------------------------------------------------------------------
--
// Средние значения и производные показатели.

// AvgReadyQueue — средняя длина очереди готовности, процессов.
func (s *Stats) AvgReadyQueue() float64 { return average(s.ReadySum, s.Now) }

// AvgIOBusy — среднее число одновременно занятых процессоров ВВ.
func (s *Stats) AvgIOBusy() float64 { return average(s.IOBusySum, s.Now) }

// CPUUtilization — полезная загрузка процессора, %.
func (s *Stats) CPUUtilization() float64 { return percent(s.CPUBusy, s.Now) }

// Throughput — производительность системы: выполненных заданий на 1000 тактов.
func (s *Stats) Throughput() float64 { return average(s.CompletedCount*1000, s.Now) }

// AvgSize — средний размер загруженных заданий, единиц памяти.
func (s *Stats) AvgSize() float64 { return average(s.SumSize, s.LoadedCount) }

// AvgCommands — среднее число команд загруженных заданий.
func (s *Stats) AvgCommands() float64 { return average(s.SumCommands, s.LoadedCount) }
```

```
// AvgIOPercent — средний процент команд ввода-вывода загруженных заданий.
func (s *Stats) AvgIOPercent() float64 { return average(s.SumIOPercent, s.LoadedCount)
}


// AvgPriority — средний базовый приоритет загруженных заданий.
func (s *Stats) AvgPriority() float64 { return average(s.SumBasePriority,
s.LoadedCount) }


// AvgWait — среднее время ожидания завершённых процессов, тактов.
func (s *Stats) AvgWait() float64 { return average(s.SumWait, s.CompletedCount) }


// AvgExec — среднее время выполнения завершённых процессов на ЦПр, тактов.
func (s *Stats) AvgExec() float64 { return average(s.SumExec, s.CompletedCount) }


// AvgIO — среднее время ввода-вывода завершённых процессов, тактов.
func (s *Stats) AvgIO() float64 { return average(s.SumIO, s.CompletedCount) }


// AvgTurnaround — среднее оборотное время завершённых процессов, тактов.
func (s *Stats) AvgTurnaround() float64 { return average(s.SumTurnaround,
s.CompletedCount) }


//
---------------------------------------------------------------------------------
--
// average возвращает sum/count (0 при отсутствии данных).
func average(sum, count int) float64 {
    if count == 0 {
        return 0
    }
    return float64(sum) / float64(count)
}


//
---------------------------------------------------------------------------------
--
// percent возвращает part/total в процентах (0 при отсутствии данных).
func percent(part, total int) float64 {
    if total == 0 {
        return 0
    }
    return float64(part) * 100 / float64(total)
}
```

## Файл internal/regulator/regulator.go:

*Регулировщик с регистрацией статистики.*

```
// Package regulator реализует регулировщик — часть ОС, отвечающую за
// состояния процессов, и включает планировщик (Planner).
//
// Только регулировщик имеет право изменять состояния процессов: остальные
// модули модели (центральный процессор, подсистема ввода-вывода) лишь
// обращаются к подпрограмме Process, сообщая о произошедшем событии.
// Планировщик решает одну задачу — выбор следующего процесса из списка
// готовности; состояния он не изменяет.
// Регулировщик переводит процесс в новое состояние и выполняет связанные
// с этим изменения общих данных модели: очереди готовности, таблицы слов
// состояний процессов, памяти и счётчика завершённых заданий.
package regulator


import (
    "os_model/internal/memory"
    "os_model/internal/process"
    "os_model/internal/stats"
)
```

```
//
-----------------------------------------------------------------------------
--
// Event — событие модели, по которому регулировщик изменяет состояние процесса.
type Event int


//
-----------------------------------------------------------------------------
--
const (
    EventLoad       Event = iota // загрузка задания: создан процесс в состоянии
«Готов»
    EventActivate                // выбор планировщика: «Готов» → «Активен»
    EventIORequest               // инициализация ВВ: «Активен» → «Инициализация IO»
    EventIOStart                 // процессор ВВ начал работу: «Инициализация IO» →
«Блокирован»
    EventIOComplete              // прерывание от ВВ: «Блокирован» → «Готов»
    EventTerminate               // завершение задания: выгрузка процесса
)


//
-----------------------------------------------------------------------------
--
// Regulator — регулировщик модели. Включает планировщик (Planner),
// выбирающий следующий процесс из очереди готовности.
type Regulator struct {
    Planner *Planner // планировщик: очередь готовности и диспетчеризация


    Memory       *memory.Allocator // супервизор памяти: освобождается при завершении
    MaxProcesses int               // число записей в таблице слов состояний процессов


    // Stats — модуль статистики: накапливает входные и выходные параметры
    // процессов (лабораторная работа 6). Может быть nil.
    Stats *stats.Stats


    Table     []*process.PSW // таблица слов состояний процессов (резидентные
процессы)
    Completed int            // число выполненных (завершённых) заданий
}


//
-----------------------------------------------------------------------------
--
// Process — основная подпрограмма регулировщика: обрабатывает событие event
// для процесса p, изменяя его состояние и общие данные модели.
// Возвращает false, если событие недопустимо для текущего состояния процесса.
func (r *Regulator) Process(p *process.PSW, event Event) bool {
    switch event {
    case EventLoad:
        // Загрузка задания: нужна свободная запись в таблице PSW.
        if !r.FreeSlot() {
            return false
        }
        p.State = process.StateReady
        r.Table = append(r.Table, p)
        r.Planner.Add(p)
        if r.Stats != nil {
            r.Stats.OnLoad(p)
        }


    case EventActivate:
        // Планировщик выбрал процесс для выполнения на процессоре.
        if p.State != process.StateReady {
            return false
        }
```

```
        p.State = process.StateActive
        r.Planner.Remove(p)

    case EventIORequest:
        // Задание инициализирует операцию ввода-вывода: слово состояния
        // процесса сохраняется, процесс переводится в «Инициализация IO».
        if p.State != process.StateActive {
            return false
        }
        p.State = process.StateInitIO

    case EventIOStart:
        // Процессор ВВ приступил к выполнению операции.
        if p.State != process.StateInitIO {
            return false
        }
        p.State = process.StateBlockedIO

    case EventIOComplete:
        // Сигнал прерывания от процессора ВВ: процесс снова готов
        // к выполнению и помещается в очередь готовности.
        if p.State != process.StateBlockedIO {
            return false
        }
        p.State = process.StateReady
        r.Planner.Add(p)

    case EventTerminate:
        // Завершение задания: выгрузка процесса.
        if p.State != process.StateActive {
            return false
        }
        r.Planner.Remove(p)
        r.Memory.Free(*p.Task)
        r.removeFromTable(p)
        p.State = process.StateNone
        r.Completed++
        if r.Stats != nil {
            r.Stats.OnComplete(p)
        }

    default:
        return false
    }

    return true
}

//
-----------------------------------------------------------------------------------
--
// Terminate — подпрограмма завершения процесса, используемая процессором
// при выполнении команды завершения задания.
func (r *Regulator) Terminate(p *process.PSW) {
    r.Process(p, EventTerminate)
}

//
-----------------------------------------------------------------------------------
--
// FreeSlot сообщает, есть ли свободная запись в таблице слов состояний процессов.
func (r *Regulator) FreeSlot() bool {
    return len(r.Table) < r.MaxProcesses
}
```

```
//
-------------------------------------------------------------------------------
--
// removeFromTable удаляет запись процесса из таблицы слов состояний процессов.
func (r *Regulator) removeFromTable(p *process.PSW) {
    for i, t := range r.Table {
        if t == p {
            r.Table = append(r.Table[:i], r.Table[i+1:]...)
            return
        }
    }
}
```

## Файл internal/os/system.go:

*Ядро модели: такт, диспетчеризация, генерация заданий.*

```
package os

import (
    "math/rand/v2"
    "time"


    "os_model/internal/cpu"
    "os_model/internal/interrupt"
    "os_model/internal/io"
    "os_model/internal/memory"
    "os_model/internal/process"
    "os_model/internal/regulator"
    "os_model/internal/stats"
)

//
-------------------------------------------------------------------------------
--
// Параметры модели по умолчанию (если соответствующие поля Config не заполнены).
const (
    DefaultMemorySize     = 8192 // общий объём моделируемой памяти
    DefaultTickDurationMs = 1    // начальная длительность такта, мс
    DefaultIOMaxDuration  = 10   // максимальная длительность ввода-вывода, тактов
)

//
-------------------------------------------------------------------------------
--
// Параметры генерации заданий (значения по умолчанию на этапе компиляции;
// на этапе 8 будут читаться из файла параметров).
const (
    TaskSizeMin = 100 // размер задания, единиц памяти
    TaskSizeMax = 500


    TaskCommandsMin = 10 // число команд задания
    TaskCommandsMax = 20


    TaskIOPercentMin = 0 // процент команд ввода-вывода
    TaskIOPercentMax = 50


    TaskBasePriorityMin = 1 // базовый приоритет
    TaskBasePriorityMax = 10
)

//
-------------------------------------------------------------------------------
--
// MaxProcesses — число записей в таблице слов состояний процессов.
```

```
const MaxProcesses = 16

//
-----------------------------------------------------------------------------------
--
// Границы скорости моделирования: от 0,1 до 1000 тактов в секунду.
const (
    minSpeed         = 0.1
    maxSpeed         = 1000.0
    speedStepPercent = 10
)

//
-----------------------------------------------------------------------------------
--
// Directive — управляющая директива оператора.
type Directive int

const (
    DirectivePause     Directive = iota // приостановить моделирование
    DirectiveResume                     // продолжить моделирование
    DirectiveSpeedUp                    // увеличить скорость
    DirectiveSpeedDown                  // уменьшить скорость
    DirectiveQuit                       // завершить моделирование
)

//
-----------------------------------------------------------------------------------
--
// LastTaskInfo — параметры последнего сгенерированного задания: выводятся
// подпрограммой индикации «Вывод параметров нового задания» (лабораторная
// работа 6). Задание может быть не загружено, если не хватило ресурсов.
type LastTaskInfo struct {
    Task   process.Task // параметры задания
    Loaded bool         // задание загружено в память
    Reason string       // причина, по которой задание не загружено
}

//
-----------------------------------------------------------------------------------
--
// System — ядро модели: связывает все компоненты и содержит главный цикл.
type System struct {
    Memory     *memory.Allocator     // супервизор памяти
    Regulator  *regulator.Regulator  // регулировщик (состояния процессов,
планировщик)
    Interrupts *interrupt.Supervisor // супервизор прерываний
    CPU        *cpu.CPU              // центральный процессор
    IO         *io.Processors        // подсистема ввода-вывода (процессоры ВВ)
    Stats      *stats.Stats          // модуль статистики (лабораторная работа 6)

    Config Config // параметры конфигурации

    // LastTask — параметры последнего сгенерированного задания.
    LastTask LastTaskInfo

    Time int // глобальное время модели в тактах

    // Управление работой модели.
    Directives chan Directive // директивы оператора
    Speed      float64        // текущая скорость модели, тактов в секунду
    Paused     bool           // моделирование приостановлено
    Quit       bool           // получена директива завершения

    // OnTick вызывается каждый такт для индикации состояния модели
```

```
    // (например, для отправки снимка в интерфейс). Может быть nil.
    OnTick func(s *System)

    nextTaskID int // счётчик идентификаторов заданий
}

//
------------------------------------------------------------------------------
--
// Init инициализирует структуры модели по значениям полей Config
// до запуска главного цикла.
func (sys *System) Init() {
    // Значения по умолчанию для незаполненной (или невалидной) конфигурации.
    if sys.Config.MemorySize <= 0 {
        sys.Config.MemorySize = DefaultMemorySize
    }
    if sys.Config.TickDurationMs <= 0 {
        sys.Config.TickDurationMs = DefaultTickDurationMs
    }
    if sys.Config.IOMaxDuration <= 0 {
        sys.Config.IOMaxDuration = DefaultIOMaxDuration
    }

    // Создание подсистем и их связывание.
    sys.Stats = &stats.Stats{}
    sys.Memory = &memory.Allocator{TotalSize: sys.Config.MemorySize}
    sys.Regulator = &regulator.Regulator{
        Planner:      &regulator.Planner{},
        Memory:       sys.Memory,
        MaxProcesses: MaxProcesses,
        Stats:        sys.Stats,
    }
    sys.Interrupts = &interrupt.Supervisor{}
    sys.IO = &io.Processors{
        Regulator:  sys.Regulator,
        Interrupts: sys.Interrupts,
    }
    sys.CPU = &cpu.CPU{
        IO:            sys.IO,
        Processes:     sys.Regulator,
        IOMaxDuration: sys.Config.IOMaxDuration,
    }

    if sys.Directives == nil {
        sys.Directives = make(chan Directive, MaxProcesses)
    }

    sys.Time = 0
    sys.nextTaskID = 0
    sys.Paused = false
    sys.Quit = false
    sys.setSpeed(1000.0 / float64(sys.Config.TickDurationMs))
}

//
------------------------------------------------------------------------------
--
// GenerateAndLoadTask — системный процесс загрузки задания: генерирует
// параметры нового задания и пытается загрузить его, если есть ресурсы
// (свободная запись в таблице слов состояний процессов и достаточная
// память). При успехе регулировщик создаёт процесс в состоянии «Готов».
// Возвращает true, если задание загружено.
func (sys *System) GenerateAndLoadTask() bool {
    // Параметры нового задания (выводятся подпрограммой индикации
    // «Вывод параметров нового задания»).
    task := sys.generateTask()
    sys.LastTask = LastTaskInfo{Task: task}
```

```
    // Проверка места в таблице слов состояний процессов.
    if !sys.Regulator.FreeSlot() {
        sys.LastTask.Reason = "нет свободной записи в таблице PSW"
        return false
    }


    // Новое задание загружается только при наличии свободной памяти.
    if !sys.Memory.CheckAvailable(task.Size) {
        sys.LastTask.Reason = "недостаточно свободной памяти"
        return false
    }


    // Присваиваем идентификатор и выделяем память для задания.
    sys.nextTaskID++
    task.ID = sys.nextTaskID
    if err := sys.Memory.Allocate(task); err != nil {
        sys.LastTask.Reason = err.Error()
        return false
    }
    sys.LastTask.Task = task
    sys.LastTask.Loaded = true


    // Порождаем процесс; состояние «Готов» устанавливает регулировщик.
    psw := &process.PSW{
        ID:              task.ID,
        Task:            &task,
        PC:              0,
        DynamicPriority: task.BasePriority,
    }


    return sys.Regulator.Process(psw, regulator.EventLoad)
}


//
-----------------------------------------------------------------------------------------
--
// HandleDirectives обрабатывает директивы оператора:
// пауза, изменение скорости моделирования, завершение работы.
func (sys *System) HandleDirectives() {
    for {
        select {
        case d := <-sys.Directives:
            sys.applyDirective(d)
        default:
            return
        }
    }
}


//
-----------------------------------------------------------------------------------------
--
// RunLoop — главный цикл модели. В каждом такте выполняется подпрограмма
// DoOneTick (алгоритм одного такта моделирования), затем обрабатываются
// директивы оператора и обновляется индикация.
func (sys *System) RunLoop() {
    // Цикл загрузки заданий для начала моделирования: загружается столько
    // заданий, сколько помещается в память и в таблицу процессов.
    for sys.GenerateAndLoadTask() {
    }


    for !sys.Quit {
        if !sys.Paused {
            sys.DoOneTick()
        } else {
```

```
            // На паузе изменения (директивы, скорость) тоже нужно показывать.
            sys.notify()
        }

        sys.HandleDirectives()

        if sys.Quit {
            sys.notify() // завершающий снимок состояния
            break
        }

        sys.sleep()
    }
}

//
-----------------------------------------------------------------------------
--
// DoOneTick — выполнение одного такта моделирования в соответствии
// с алгоритмом одного такта модели ОС:
//
//    А  — инициализация цикла: выбрать процесс, сделать его активным
//         и восстановить слово состояния процесса;
//    Сб — сбор статистики за такт (времена процессов, занятость ЦПр,
//         длины очередей) — лабораторная работа 6;
//    Б1 — моделирование ЦПр: выполнить очередную команду активного процесса;
//    Б2 — моделирование процессоров ВВ: один такт операций ввода-вывода
//         и обработка прерываний от процессоров ВВ;
//    Б3 — моделирование прерываний по времени (только для алгоритма RR;
//         вариант 11 использует относительные приоритеты без вытеснения,
//         поэтому шаг отсутствует);
//    Б4 — проверка достигнутого состояния системы и вызов планировщика;
//    Б5 — обновление баз данных модели ОС;
//    Б6 — отображение изменений в состоянии системы.
func (sys *System) DoOneTick() {
    sys.Time++

    // А. Инициализация цикла.
    if sys.CPU.State == cpu.StateIdle {
        sys.dispatchProcess()
    }

    // Сб. Учёт статистики за текущий такт: состояние фиксируется после
    // диспетчеризации и до выполнения команды.
    sys.collectStats()

    // Б1. Моделирование центрального процессора.
    sys.CPU.Tick()

    // Б2. Моделирование процессоров ввода-вывода и обработка прерываний.
    sys.IO.TickIO(sys.Time)
    sys.HandleInterrupts()

    // Б3. Прерывания по времени не моделируются (см. комментарий выше).

    // Б4. Проверка достигнутого состояния системы.
    sys.CheckSystemState()

    // Б5. Обновление баз данных модели ОС.
    sys.Regulator.Planner.UpdatePriorities()

    // Б6. Отображение изменений в состоянии системы.
    sys.notify()
}
```

```
//
-------------------------------------------------------------------------------
--
// dispatchProcess — выбор планировщиком следующего процесса и передача его
// на выполнение: регулировщик переводит процесс в состояние «Активен»,
// процессор восстанавливает слово состояния процесса.
func (sys *System) dispatchProcess() bool {
    p := sys.Regulator.Planner.SelectNext()
    if p == nil {
        return false
    }

    if !sys.Regulator.Process(p, regulator.EventActivate) {
        return false
    }

    sys.Stats.OnDispatch(p.ID)
    sys.CPU.ContextSwitch(p)
    return true
}

//
-------------------------------------------------------------------------------
--
// collectStats — подпрограмма сбора статистики за такт (лабораторная
// работа 6): занятость центрального процессора, длины очередей и времена
// процессов по их состояниям.
func (sys *System) collectStats() {
    sys.Stats.Collect(
        sys.Time,
        sys.CPU.State == cpu.StateWorking,
        sys.Regulator.Table,
        len(sys.Regulator.Planner.ReadyQueue),
        len(sys.IO.Operations),
    )
}

//
-------------------------------------------------------------------------------
--
// CheckSystemState — проверка достигнутого состояния системы (шаг Б4).
//
// Для всех процессоров: если резидентный процесс перестал быть активным
// (заблокирован или завершён), процессор освобождается; если процессор
// находится в состоянии «Ожидание», а список готовности не пуст, вызывается
// планировщик для выбора следующего процесса. Если готовых процессов нет,
// процессор переходит в состояние «Ожидание», а ОС порождает системный
// процесс загрузки нового задания, если есть ресурсы.
func (sys *System) CheckSystemState() {
    // Резидентный процесс мог перестать быть активным: его состояние
    // изменил регулировщик по событию ВВ или завершения.
    if p := sys.CPU.ActiveProcess; p != nil && p.State != process.StateActive {
        sys.CPU.Release()
    }

    if sys.CPU.State != cpu.StateIdle {
        return
    }

    // Список готовности не пуст — планировщик выбирает следующий процесс.
    if len(sys.Regulator.Planner.ReadyQueue) > 0 {
        sys.dispatchProcess()
        return
    }
```

```
    // Готовых процессов нет: ЦПр «Ожидает», ОС может загрузить одно новое
    // задание (место в таблице PSW и достаточная память проверяются
    // в GenerateAndLoadTask).
    if sys.GenerateAndLoadTask() {
        sys.dispatchProcess()
    }
}

//
------------------------------------------------------------------------------
--
// HandleInterrupts — супервизор прерываний: обрабатывает накопленные сигналы.
// Обработка сводится к вызову регулировщика, который изменяет состояния
// процессов (прерывание от процессора ВВ переводит процесс в «Готов»).
func (sys *System) HandleInterrupts() {
    for _, signal := range sys.Interrupts.ExtractAll() {
        switch signal.Type {
        case interrupt.IOComplete:
            sys.Regulator.Process(signal.Process, regulator.EventIOComplete)
        }
    }
}

//
------------------------------------------------------------------------------
--
// generateTask генерирует параметры нового задания в заданных диапазонах
// (без идентификатора: идентификатор присваивается загруженному заданию).
func (sys *System) generateTask() process.Task {
    return process.Task{
        Size:          randomInRange(TaskSizeMin, TaskSizeMax),
        TotalCommands: randomInRange(TaskCommandsMin, TaskCommandsMax),
        IOPercent:     randomInRange(TaskIOPercentMin, TaskIOPercentMax),
        BasePriority:  randomInRange(TaskBasePriorityMin, TaskBasePriorityMax),
    }
}

//
------------------------------------------------------------------------------
--
// applyDirective выполняет одну директиву оператора.
func (sys *System) applyDirective(d Directive) {
    switch d {
    case DirectivePause:
        sys.Paused = true
    case DirectiveResume:
        sys.Paused = false
    case DirectiveSpeedUp:
        sys.changeSpeed(speedStepPercent)
    case DirectiveSpeedDown:
        sys.changeSpeed(-speedStepPercent)
    case DirectiveQuit:
        sys.Quit = true
    }
}

//
------------------------------------------------------------------------------
--
// changeSpeed изменяет скорость модели на заданное число процентов.
func (sys *System) changeSpeed(percent int) {
    sys.setSpeed(sys.Speed * float64(100+percent) / 100)
}

//
------------------------------------------------------------------------------
--
// setSpeed устанавливает скорость модели (тактов в секунду)
```

```
// в допустимых пределах.
func (sys *System) setSpeed(speed float64) {
    if speed < minSpeed {
        speed = minSpeed
    }
    if speed > maxSpeed {
        speed = maxSpeed
    }
    sys.Speed = speed
}

//
-------------------------------------------------------------------------------
--
// notify вызывает обработчик индикации, если он установлен.
func (sys *System) notify() {
    if sys.OnTick != nil {
        sys.OnTick(sys)
    }
}

//
-------------------------------------------------------------------------------
--
// sleep выполняет программную задержку в соответствии со скоростью модели.
func (sys *System) sleep() {
    if sys.Speed <= 0 {
        return
    }
    time.Sleep(time.Duration(float64(time.Second) / sys.Speed))
}

//
-------------------------------------------------------------------------------
--
// randomInRange возвращает случайное число из диапазона [min, max].
func randomInRange(min, max int) int {
    if max <= min {
        return min
    }
    return min + rand.IntN(max-min+1)
}

//
-------------------------------------------------------------------------------
--
```

## Файл internal/os/snapshot.go:

*Снимок состояния для индикации.*

```
package os

import (
    "os_model/internal/interrupt"
    "os_model/internal/io"
    "os_model/internal/process"
)

//
-------------------------------------------------------------------------------
--
// ProcessInfo — данные одного процесса для индикации.
type ProcessInfo struct {
    ID              int `json:"id"`
    Size            int `json:"size"`
```

```
    PC              int `json:"pc"`
    TotalCommands   int `json:"totalCommands"`
    IOPercent       int `json:"ioPercent"`
    State           int `json:"state"`
    DynamicPriority int `json:"dynamicPriority"`
    BasePriority    int `json:"basePriority"`


    // Времена процесса, тактов (лабораторная работа 6).
    WaitTicks  int `json:"waitTicks"`  // ожидание в очереди готовности
    ExecTicks  int `json:"execTicks"`  // выполнение на ЦПр
    IOTicks    int `json:"ioTicks"`    // «Инициализация IO» и «Блокирован»
    Turnaround int `json:"turnaround"` // оборотное время от загрузки
}

//
------------------------------------------------------------------------------
--
// OperationInfo — данные одной операции ввода-вывода (занятого процессора ВВ).
type OperationInfo struct {
    ProcessorID int `json:"processorId"` // номер процессора ВВ
    ProcessID   int `json:"processId"`   // обслуживаемый процесс
    State       int `json:"state"`       // «Инициализация IO» или «Блокирован»
    TicksLeft   int `json:"ticksLeft"`   // осталось тактов
    TotalTicks  int `json:"totalTicks"`  // полная длительность операции
}

//
------------------------------------------------------------------------------
--
// InterruptInfo — данные сигнала прерывания для индикации.
type InterruptInfo struct {
    Type      int `json:"type"`      // тип прерывания
    Time      int `json:"time"`      // такт модели
    ProcessID int `json:"processId"` // процесс, которого касается прерывание
    SourceID  int `json:"sourceId"`  // номер источника (процессора ВВ)
}

//
------------------------------------------------------------------------------
--
// TaskParams — диапазоны генерации заданий (входные параметры модели).
type TaskParams struct {
    SizeMin      int `json:"sizeMin"`
    SizeMax      int `json:"sizeMax"`
    CommandsMin  int `json:"commandsMin"`
    CommandsMax  int `json:"commandsMax"`
    IOPercentMin int `json:"ioPercentMin"`
    IOPercentMax int `json:"ioPercentMax"`
    PriorityMin  int `json:"priorityMin"`
    PriorityMax  int `json:"priorityMax"`
}

//
------------------------------------------------------------------------------
--
// TaskInfo — параметры нового (последнего сгенерированного) задания.
type TaskInfo struct {
    ID            int    `json:"id"`
    Size          int    `json:"size"`
    TotalCommands int    `json:"totalCommands"`
    IOPercent     int    `json:"ioPercent"`
    BasePriority  int    `json:"basePriority"`
    Loaded        bool   `json:"loaded"` // задание загружено в память
    Reason        string `json:"reason"` // причина, если загрузить не удалось
}
```

```
//
--------------------------------------------------------------------------------
--
// StatsInfo — статистика модели для индикации (лабораторная работа 6).
type StatsInfo struct {
    // Центральный процессор.
    CPUBusy        int     `json:"cpuBusy"`        // тактов процессор выполнял
команды
    CPUIdle        int     `json:"cpuIdle"`        // тактов процессор простаивал
    CPUUtilization float64 `json:"cpuUtilization"` // полезная загрузка ЦПр, %


    // Длины очередей.
    ReadyLen  int     `json:"readyLen"`  // текущая длина очереди готовности
    ReadyAvg  float64 `json:"readyAvg"`  // средняя длина очереди готовности
    ReadyMax  int     `json:"readyMax"`  // максимальная длина очереди
    IOBusy    int     `json:"ioBusy"`    // занятых процессоров ВВ сейчас
    IOBusyAvg float64 `json:"ioBusyAvg"` // среднее число занятых процессоров ВВ
    IOBusyMax int     `json:"ioBusyMax"` // максимум занятых процессоров ВВ


    // Производительность системы: выполненных заданий на 1000 тактов.
    Throughput float64 `json:"throughput"`


    // Последовательность выполнения процессов на ЦПр (до 16 номеров).
    ExecHistory []int `json:"execHistory"`


    // Средние входные параметры загруженных заданий.
    LoadedCount  int     `json:"loadedCount"`
    AvgSize      float64 `json:"avgSize"`
    AvgCommands  float64 `json:"avgCommands"`
    AvgIOPercent float64 `json:"avgIoPercent"`
    AvgPriority  float64 `json:"avgPriority"`


    // Средние выходные параметры завершённых процессов.
    CompletedCount int     `json:"completedCount"`
    AvgWait        float64 `json:"avgWait"`       // ожидание, тактов
    AvgExec        float64 `json:"avgExec"`       // выполнение на ЦПр, тактов
    AvgIO          float64 `json:"avgIo"`         // ввод-вывод, тактов
    AvgTurnaround  float64 `json:"avgTurnaround"` // оборотное время, тактов
}


//
--------------------------------------------------------------------------------
--
// Snapshot — снимок состояния модели для индикации в интерфейсе.
type Snapshot struct {
    Time      int     `json:"time"`
    Speed     float64 `json:"speed"`
    Paused    bool    `json:"paused"`
    Quit      bool    `json:"quit"`
    Generated int     `json:"generated"`
    Completed int     `json:"completed"`


    MemoryTotal int `json:"memoryTotal"`
    MemoryUsed  int `json:"memoryUsed"`


    CPUState        int `json:"cpuState"`        // 0 — ожидание, 1 — работа
    ActiveID        int `json:"activeId"`        // 0 — нет активного процесса
    CommandType     int `json:"commandType"`     // 0 — вычислительная, 1 — IO, 2 —
завершение
    CommandCode     int `json:"commandCode"`     // код вычислительной операции
    CommandAddr1    int `json:"commandAddr1"`    // адрес первого операнда
    CommandAddr2    int `json:"commandAddr2"`    // адрес второго операнда
    CommandDuration int `json:"commandDuration"` // длительность операции IO, тактов


    // Прерывания: счётчик обработанных сигналов и журнал последних.
    InterruptsHandled int             `json:"interruptsHandled"`
```

```
    InterruptLog      []InterruptInfo `json:"interruptLog"`


    MaxProcesses int `json:"maxProcesses"`


    Ready      []ProcessInfo   `json:"ready"`      // очередь готовности
    Operations []OperationInfo `json:"operations"` // занятые процессоры ВВ
    Processes  []ProcessInfo   `json:"processes"`  // таблица PSW


    TaskParams TaskParams `json:"taskParams"`
    LastTask   TaskInfo   `json:"lastTask"` // параметры нового задания
    Stats      StatsInfo  `json:"stats"`    // статистика модели (лаб. 6)
}


//
----------------------------------------------------------------------------------
--
// Snapshot формирует снимок текущего состояния модели.
func (sys *System) Snapshot() Snapshot {
    snap := Snapshot{
        Time:              sys.Time,
        Speed:             sys.Speed,
        Paused:            sys.Paused,
        Quit:              sys.Quit,
        Generated:         sys.nextTaskID,
        Completed:         sys.Regulator.Completed,
        MemoryTotal:       sys.Memory.TotalSize,
        MemoryUsed:        sys.Memory.UsedSize,
        CPUState:          int(sys.CPU.State),
        InterruptsHandled: sys.Interrupts.Handled,
        MaxProcesses:      MaxProcesses,
        TaskParams: TaskParams{
            SizeMin:      TaskSizeMin,
            SizeMax:      TaskSizeMax,
            CommandsMin:  TaskCommandsMin,
            CommandsMax:  TaskCommandsMax,
            IOPercentMin: TaskIOPercentMin,
            IOPercentMax: TaskIOPercentMax,
            PriorityMin:  TaskBasePriorityMin,
            PriorityMax:  TaskBasePriorityMax,
        },
    }


    // Команда актуальна только при активном процессе.
    if p := sys.CPU.ActiveProcess; p != nil {
        snap.ActiveID = p.ID
        snap.CommandType = int(sys.CPU.CurrentCommand.Type)
        snap.CommandCode = int(sys.CPU.CurrentCommand.Code)
        snap.CommandAddr1 = sys.CPU.CurrentCommand.Address1
        snap.CommandAddr2 = sys.CPU.CurrentCommand.Address2
        snap.CommandDuration = sys.CPU.CurrentCommand.Duration
    }


    snap.LastTask = TaskInfo{
        ID:            sys.LastTask.Task.ID,
        Size:          sys.LastTask.Task.Size,
        TotalCommands: sys.LastTask.Task.TotalCommands,
        IOPercent:     sys.LastTask.Task.IOPercent,
        BasePriority:  sys.LastTask.Task.BasePriority,
        Loaded:        sys.LastTask.Loaded,
        Reason:        sys.LastTask.Reason,
    }
    snap.Stats = sys.statsInfo()


    snap.Ready = processInfos(sys.Regulator.Planner.ReadyQueue, sys.Time)
    snap.Operations = operationInfos(sys.IO.Operations)
    snap.Processes = processInfos(sys.Regulator.Table, sys.Time)
    snap.InterruptLog = interruptInfos(sys.Interrupts.Log)
```

```
    return snap
}

//
-------------------------------------------------------------------------------
--
// statsInfo собирает статистику модели в данные для индикации.
func (sys *System) statsInfo() StatsInfo {
    return StatsInfo{
        CPUBusy:        sys.Stats.CPUBusy,
        CPUIdle:        sys.Stats.CPUIdle,
        CPUUtilization: sys.Stats.CPUUtilization(),
        ReadyLen:       len(sys.Regulator.Planner.ReadyQueue),
        ReadyAvg:       sys.Stats.AvgReadyQueue(),
        ReadyMax:       sys.Stats.ReadyMax,
        IOBusy:         len(sys.IO.Operations),
        IOBusyAvg:      sys.Stats.AvgIOBusy(),
        IOBusyMax:      sys.Stats.IOBusyMax,
        Throughput:     sys.Stats.Throughput(),
        ExecHistory:    sys.Stats.History(),
        LoadedCount:    sys.Stats.LoadedCount,
        AvgSize:        sys.Stats.AvgSize(),
        AvgCommands:    sys.Stats.AvgCommands(),
        AvgIOPercent:   sys.Stats.AvgIOPercent(),
        AvgPriority:    sys.Stats.AvgPriority(),
        CompletedCount: sys.Stats.CompletedCount,
        AvgWait:        sys.Stats.AvgWait(),
        AvgExec:        sys.Stats.AvgExec(),
        AvgIO:          sys.Stats.AvgIO(),
        AvgTurnaround:  sys.Stats.AvgTurnaround(),
    }
}

//
-------------------------------------------------------------------------------
--
// processInfos преобразует список PSW в данные для индикации.
// now — текущий такт модели: по нему вычисляется оборотное время.
func processInfos(psws []*process.PSW, now int) []ProcessInfo {
    infos := make([]ProcessInfo, 0, len(psws))
    for _, p := range psws {
        infos = append(infos, ProcessInfo{
            ID:              p.ID,
            Size:            p.Task.Size,
            PC:              p.PC,
            TotalCommands:   p.Task.TotalCommands,
            IOPercent:       p.Task.IOPercent,
            State:           int(p.State),
            DynamicPriority: p.DynamicPriority,
            BasePriority:    p.Task.BasePriority,
            WaitTicks:       p.WaitTicks,
            ExecTicks:       p.ExecTicks,
            IOTicks:         p.IOTicks,
            Turnaround:      now - p.LoadedAt,
        })
    }
    return infos
}

//
-------------------------------------------------------------------------------
--
// operationInfos преобразует операции ввода-вывода в данные для индикации.
func operationInfos(operations []*io.Operation) []OperationInfo {
    infos := make([]OperationInfo, 0, len(operations))
    for _, op := range operations {
        infos = append(infos, OperationInfo{
            ProcessorID: op.ProcessorID,
            ProcessID:   op.Process.ID,
```

```
            State:       int(op.Process.State),
            TicksLeft:   op.TicksLeft,
            TotalTicks:  op.TotalTicks,
        })
    }
    return infos
}

//
-----------------------------------------------------------------------------------
--
// interruptInfos преобразует журнал прерываний в данные для индикации.
func interruptInfos(signals []interrupt.Signal) []InterruptInfo {
    infos := make([]InterruptInfo, 0, len(signals))
    for _, s := range signals {
        info := InterruptInfo{
            Type:     int(s.Type),
            Time:     s.Time,
            SourceID: s.SourceID,
        }
        if s.Process != nil {
            info.ProcessID = s.Process.ID
        }
        infos = append(infos, info)
    }
    return infos
}
```

## Файл frontend/dist/index.html:

*Разметка единого экрана индикации.*

```
<!DOCTYPE html>
<html lang="ru">
<head>
  <meta charset="UTF-8" />
  <title>Модель ОС — вариант 11</title>
  <link rel="stylesheet" href="style.css" />
</head>
<body>
  <div class="app">


    <!-- Постоянная информация: заголовок, состояние модели, язык управления -->
    <header class="masthead">
      <div class="masthead-title">
        <h1>Модель операционной системы</h1>
        <p>Вариант 11 — относительные и динамические приоритеты</p>
      </div>


      <dl class="run-state">
        <div>
          <dt>Модельное время</dt>
          <dd class="mono" id="m-time">0</dd>
          <span class="unit">такт</span>
        </div>
        <div>
          <dt>Скорость</dt>
          <dd class="mono" id="m-speed">10</dd>
          <span class="unit">такт/с</span>
        </div>
        <div>
          <dt>Состояние</dt>
          <dd class="state st-active" id="m-status">Работа</dd>
        </div>
      </dl>


      <div class="controls">
        <button type="button" id="btn-pause">Пауза</button>
```

```
        <button type="button" id="btn-slower" title="Уменьшить скорость
(−)">−</button>
        <button type="button" id="btn-faster" title="Увеличить скорость
(+)">+</button>
        <button type="button" id="btn-quit" class="danger" title="Завершить
моделирование (Q)">Завершить</button>
      </div>


      <p class="control-help">
        Управление: <kbd>Space</kbd> — пауза / продолжить,
        <kbd>+</kbd> <kbd>−</kbd> — скорость ± 10 %,
        <kbd>Q</kbd> — завершить моделирование
      </p>
    </header>


    <main class="grid">


      <!-- Вывод параметров системы -->
      <section class="panel span-5">
        <header class="panel-head">
          <h2>Параметры системы</h2>
        </header>
        <div class="panel-body">
          <dl class="kv two-col">
            <div><dt>Загружено заданий</dt><dd class="mono"
id="sys-generated">0</dd></div>
            <div><dt>Выполнено заданий</dt><dd class="mono"
id="sys-completed">0</dd></div>
            <div><dt>Таблица PSW</dt><dd class="mono" id="sys-table">0 / 16</dd></div>
            <div><dt>Прерываний обработано</dt><dd class="mono"
id="sys-int">0</dd></div>
            <div><dt>Очередь готовности, текущая / средняя / макс.</dt><dd
class="mono" id="sys-ready">0 / 0 / 0</dd></div>
            <div><dt>Процессоры ВВ, занято / среднее / макс.</dt><dd class="mono"
id="sys-io">0 / 0 / 0</dd></div>
            <div><dt>Производительность, заданий на 1000 тактов</dt><dd class="mono"
id="sys-throughput">0</dd></div>
            <div><dt>Свободная память</dt><dd class="mono" id="mem-free">0</dd></div>
          </dl>


          <div class="memory">
            <div class="meter"><i id="mem-bar"></i></div>
            <span class="memory-text mono"><span id="mem-used">0</span> / <span
id="mem-total">0</span> ед. занято, <span id="mem-percent">0 %</span></span>
          </div>
        </div>
      </section>


      <!-- Вывод параметров центрального процессора -->
      <section class="panel span-4">
        <header class="panel-head">
          <h2>Параметры центрального процессора</h2>
          <span class="state st-none" id="cpu-state">Ожидание</span>
        </header>
        <div class="panel-body">
          <dl class="kv">
            <div><dt>Активный процесс</dt><dd class="mono"
id="cpu-active">—</dd></div>
            <div><dt>Выполняемая команда</dt><dd class="mono"
id="cpu-command">—</dd></div>
            <div><dt>Занят / простой, тактов</dt><dd class="mono" id="cpu-busy">0 /
0</dd></div>
            <div><dt>Полезная загрузка</dt><dd class="mono" id="cpu-load">0
%</dd></div>
          </dl>
          <div class="meter"><i id="cpu-load-bar"></i></div>
        </div>
      </section>
```

```
      <!-- Вывод параметров нового задания -->
      <section class="panel span-3">
        <header class="panel-head">
          <h2>Параметры нового задания</h2>
          <span class="state st-none" id="task-status">—</span>
        </header>
        <div class="panel-body">
          <dl class="kv two-col">
            <div><dt>Номер</dt><dd class="mono" id="task-id">—</dd></div>
            <div><dt>Приоритет</dt><dd class="mono" id="task-priority">—</dd></div>
            <div><dt>Размер</dt><dd class="mono" id="task-size">—</dd></div>
            <div><dt>Команд</dt><dd class="mono" id="task-commands">—</dd></div>
            <div><dt>Команд ВВ</dt><dd class="mono" id="task-io">—</dd></div>
          </dl>
          <p class="hint hint-error" id="task-reason" hidden></p>
          <p class="hint" id="task-ranges">—</p>
        </div>
      </section>


      <!-- Вывод параметров процессов (все процессы на одном окне) -->
      <section class="panel span-12">
        <header class="panel-head">
          <h2>Параметры процессов (таблица PSW)</h2>
          <span class="badge mono" id="proc-count">0 / 16</span>
        </header>
        <div class="panel-body">
          <div class="table-wrap">
            <table class="table" id="proc-table">
              <thead>
                <tr>
                  <th>№</th><th>Состояние</th><th>PC</th><th>Команд</th><th>Размер</
th>
                  <th>ВВ, %</th><th>Приоритет база → динамика</th>
                  <th>Ожидание</th><th>ЦПр</th><th>ВВ</th><th>Оборот</th>
                </tr>
              </thead>
              <tbody></tbody>
            </table>
          </div>
          <div class="empty" id="proc-empty">Нет загруженных процессов</div>
        </div>
      </section>


      <!-- Вывод средних значений параметров процессов -->
      <section class="panel span-5">
        <header class="panel-head">
          <h2>Средние значения параметров процессов</h2>
        </header>
        <div class="panel-body">
          <div class="avg-block">
            <h3>Входные параметры <span class="badge mono" id="avg-loaded-count">0
заданий</span></h3>
            <dl class="kv two-col">
              <div><dt>Размер</dt><dd class="mono" id="avg-size">0</dd></div>
              <div><dt>Команд</dt><dd class="mono" id="avg-commands">0</dd></div>
              <div><dt>Команд ВВ</dt><dd class="mono" id="avg-io-percent">0
%</dd></div>
              <div><dt>Базовый приоритет</dt><dd class="mono"
id="avg-priority">0</dd></div>
            </dl>
          </div>
          <div class="avg-block">
            <h3>Выходные параметры <span class="badge mono" id="avg-completed-count">0
процессов</span></h3>
            <dl class="kv two-col">
              <div><dt>Время ожидания, тактов</dt><dd class="mono"
id="avg-wait">0</dd></div>
```

```
              <div><dt>Время выполнения, тактов</dt><dd class="mono" id="avg-
exec">0</dd></div>
              <div><dt>Время ВВ, тактов</dt><dd class="mono" id="avg-io">0</dd></div>
              <div><dt>Оборотное время, тактов</dt><dd class="mono" id="avg-
turnaround">0</dd></div>
            </dl>
          </div>
        </div>
      </section>

      <!-- Вывод последовательности выполнения процессов на процессоре -->
      <section class="panel span-7">
        <header class="panel-head">
          <h2>Последовательность выполнения процессов на процессоре</h2>
          <span class="badge mono" id="history-count">0 / 16</span>
        </header>
        <div class="panel-body">
          <div class="history" id="exec-history"></div>
          <p class="hint">
            Номера процессов в порядке их выбора планировщиком на ЦПр; хранятся
            16 последних. Повторный номер — процесс вернулся на процессор после
            ввода-вывода или ожидания.
          </p>
        </div>
      </section>

      <!-- Очередь готовности -->
      <section class="panel span-5">
        <header class="panel-head">
          <h2>Очередь готовности</h2>
          <span class="badge mono" id="ready-count">0</span>
        </header>
        <div class="panel-body">
          <div class="table-wrap">
            <table class="table" id="ready-table">
              <thead>

<tr><th>№</th><th>Состояние</th><th>PC</th><th>Размер</th><th>Динамический
приоритет</th></tr>
              </thead>
              <tbody></tbody>
            </table>
          </div>
          <div class="empty" id="ready-empty">Очередь пуста</div>
        </div>
      </section>

      <!-- Процессоры ввода-вывода -->
      <section class="panel span-4">
        <header class="panel-head">
          <h2>Процессоры ввода-вывода</h2>
          <span class="badge mono" id="io-count">0</span>
        </header>
        <div class="panel-body">
          <div class="table-wrap">
            <table class="table" id="io-table">
              <thead>
                <tr><th>Процессор</th><th>Процесс</th><th>Состояние</th><th>Тактов</
th><th>Прогресс</th></tr>
              </thead>
              <tbody></tbody>
            </table>
          </div>
          <div class="empty" id="io-empty">Нет активных операций ввода-вывода</div>
        </div>
      </section>

      <!-- Прерывания -->
      <section class="panel span-3">
```

```
        <header class="panel-head">
          <h2>Прерывания</h2>
          <span class="badge mono" id="int-count">0</span>
        </header>
        <div class="panel-body">
          <div class="table-wrap">
            <table class="table" id="int-table">
              <thead>
                <tr><th>Такт</th><th>Тип</th><th>Источник</th><th>Процесс</th></tr>
              </thead>
              <tbody></tbody>
            </table>
          </div>
          <div class="empty" id="int-empty">Прерываний ещё не было</div>
        </div>
      </section>


    </main>


    <footer class="footer">
      <span>Диспетчеризация: <code>P = argmax(DynamicPriority)</code>, выбор только
при освобождении ЦПр · приоритет в очереди: <code>DynamicPriority += Size</code>
каждый такт</span>
      <span>Полезная загрузка ЦПр = <code>T<sub>занят</sub> / T</code> ·
производительность = <code>N<sub>выполнено</sub> / T · 1000</code> · оборотное время =
<code>t<sub>завершения</sub> − t<sub>загрузки</sub></code></span>
    </footer>
  </div>


  <script src="app.js"></script>
</body>
</html>
```

## Файл frontend/dist/app.js:

*Подпрограммы отрисовки интерфейса.*

```
// ============================================================================
// Модель ОС — интерфейс (вариант 11).
//
// Обмен с ядром идёт через события Wails:
//   os:update    — снимок состояния из ядра (Go → JS);
//   os:directive — управляющие директивы оператора (JS → Go).
//
// Разделы индикации соответствуют лабораторной работе 6: параметры системы,
// параметры нового задания, параметры центрального процессора, параметры
// процессов, последовательность выполнения процессов и средние значения.
// ============================================================================


"use strict";


const STATE_NAMES = ["Отсутствует", "Готов", "Активен", "Инициализация IO",
"Блокирован"];
const STATE_CLASSES = ["st-none", "st-ready", "st-active", "st-io-init", "st-
blocked"];
const CPU_NAMES = ["Ожидание", "Работа"];
const OP_NAMES = ["+", "−", "×", "÷"];
const INTERRUPT_NAMES = ["окончание ВВ", "таймер"];


const $ = (id) => document.getElementById(id);


let paused = false;
let finished = false;


// ----------------------------------------------------------------------------
```

```
// Отправка директив в ядро
// ---------------------------------------------------------------------------


function send(name) {
  if (window.runtime) {
    window.runtime.EventsEmit("os:directive", name);
  }
}


$("btn-pause").addEventListener("click", () => send(paused ? "resume" : "pause"));
$("btn-slower").addEventListener("click", () => send("speed-down"));
$("btn-faster").addEventListener("click", () => send("speed-up"));
$("btn-quit").addEventListener("click", () => {
  if (!finished && confirm("Завершить моделирование?")) {
    send("quit");
  }
});


window.addEventListener("keydown", (event) => {
  if (finished || event.repeat) {
    return;
  }


  if (event.code === "Space") {
    event.preventDefault();
    send(paused ? "resume" : "pause");
    return;
  }


  if (event.key === "+" || event.key === "=") {
    send("speed-up");
    return;
  }


  if (event.key === "-" || event.key === "_") {
    send("speed-down");
    return;
  }


  if ("qQйЙ".includes(event.key) && confirm("Завершить моделирование?")) {
    send("quit");
  }
});


// ---------------------------------------------------------------------------
// Вспомогательные функции отображения
// ---------------------------------------------------------------------------


function setText(id, value) {
  const el = $(id);
  if (el) {
    el.textContent = value;
  }
}


// fmtInt — целое число в русской локали; fmt1 — число с одним знаком после запятой.
function fmtInt(value) {
  return Number(value ?? 0).toLocaleString("ru-RU", { maximumFractionDigits: 0 });
}


function fmt1(value) {
  return Number(value ?? 0).toLocaleString("ru-RU", {
    minimumFractionDigits: 1,
    maximumFractionDigits: 1,
  });
```

```
}

function stateMark(state) {
  const name = STATE_NAMES[state] ?? "?";
  const cls = STATE_CLASSES[state] ?? "st-none";
  return `<span class="state ${cls}">${name}</span>`;
}

function commandText(snapshot) {
  if (!snapshot.activeId) {
    return "—";
  }

  switch (snapshot.commandType) {
    case 0:
      return `вычислительная: ${OP_NAMES[snapshot.commandCode] ?? "?"} (A1=$
{snapshot.commandAddr1}, A2=${snapshot.commandAddr2})`;
    case 1:
      return `ввод-вывод: ${snapshot.commandDuration} такт.`;
    case 2:
      return "завершение задания";
    default:
      return "—";
  }
}

// renderRows отрисовывает таблицу или заглушку «нет данных», чтобы панель
// не «дёргалась» при изменении числа строк.
function renderRows(tableId, rows, emptyId, rowBuilder) {
  const tbody = $(tableId).querySelector("tbody");
  tbody.innerHTML = rows.map(rowBuilder).join("");

  $(tableId).parentElement.style.display = rows.length ? "" : "none";
  $(emptyId).style.display = rows.length ? "none" : "flex";
}

// ---------------------------------------------------------------------------
// Разделы индикации
// ---------------------------------------------------------------------------

function renderHeader(snapshot) {
  paused = snapshot.paused;
  finished = snapshot.quit;

  setText("m-time", fmtInt(snapshot.time));
  setText("m-speed", fmt1(snapshot.speed));

  const status = $("m-status");
  if (snapshot.quit) {
    status.textContent = "Завершено";
    status.className = "state st-none";
  } else if (snapshot.paused) {
    status.textContent = "Пауза";
    status.className = "state st-blocked";
  } else {
    status.textContent = "Работа";
    status.className = "state st-active";
  }

  $("btn-pause").textContent = snapshot.paused ? "Продолжить" : "Пауза";
  $("btn-pause").disabled = snapshot.quit;
  $("btn-slower").disabled = snapshot.quit;
  $("btn-faster").disabled = snapshot.quit;
  $("btn-quit").disabled = snapshot.quit;
}
```

```
function renderSystem(snapshot) {
  const stats = snapshot.stats ?? {};
  const free = snapshot.memoryTotal - snapshot.memoryUsed;
  const percent = snapshot.memoryTotal
    ? Math.round((snapshot.memoryUsed / snapshot.memoryTotal) * 100)
    : 0;


  setText("sys-generated", fmtInt(snapshot.generated));
  setText("sys-completed", fmtInt(snapshot.completed));
  setText("sys-table", `${snapshot.processes.length} / ${snapshot.maxProcesses}`);
  setText("sys-int", fmtInt(snapshot.interruptsHandled));
  setText(
    "sys-ready",
    `${fmtInt(stats.readyLen)} / ${fmt1(stats.readyAvg)} / ${fmtInt(stats.readyMax)}`
  );
  setText(
    "sys-io",
    `${fmtInt(stats.ioBusy)} / ${fmt1(stats.ioBusyAvg)} / ${fmtInt(stats.ioBusyMax)}`
  );
  setText("sys-throughput", fmt1(stats.throughput));


  $("mem-bar").style.width = percent + "%";
  setText("mem-percent", percent + " %");
  setText("mem-used", fmtInt(snapshot.memoryUsed));
  setText("mem-free", fmtInt(free));
  setText("mem-total", fmtInt(snapshot.memoryTotal));
}


function renderCPU(snapshot) {
  const stats = snapshot.stats ?? {};


  const cpuState = $("cpu-state");
  cpuState.textContent = CPU_NAMES[snapshot.cpuState] ?? "—";
  cpuState.className = "state " + (snapshot.cpuState === 1 ? "st-active" : "st-none");


  setText("cpu-active", snapshot.activeId ? `#${snapshot.activeId}` : "—");
  setText("cpu-command", commandText(snapshot));
  setText("cpu-busy", `${fmtInt(stats.cpuBusy)} / ${fmtInt(stats.cpuIdle)}`);
  setText("cpu-load", fmt1(stats.cpuUtilization) + " %");


  const load = Math.max(0, Math.min(100, stats.cpuUtilization ?? 0));
  $("cpu-load-bar").style.width = load + "%";
}


function renderTask(snapshot) {
  const task = snapshot.lastTask ?? {};
  // Задание может быть ещё не загружено (нет места в таблице PSW или
  // памяти) — тогда выводится заявка без номера и со статусом ошибки.
  const hasTask = task.size > 0 || task.totalCommands > 0;
  const status = $("task-status");


  if (!hasTask) {
    status.textContent = "—";
    status.className = "state st-none";
    status.removeAttribute("title");
  } else if (task.loaded) {
    status.textContent = "Загружено";
    status.className = "state st-active";
    status.removeAttribute("title");
  } else {
    status.textContent = "Не загружено";
    status.className = "state st-blocked";
    status.title = task.reason ?? "";
  }
```

```
  const reason = $("task-reason");
  if (hasTask && !task.loaded && task.reason) {
    reason.textContent = "Причина: " + task.reason;
    reason.hidden = false;
  } else {
    reason.textContent = "";
    reason.hidden = true;
  }

  setText("task-id", task.id ? `#${task.id}` : "заявка");
  setText("task-size", hasTask ? fmtInt(task.size) : "—");
  setText("task-commands", hasTask ? fmtInt(task.totalCommands) : "—");
  setText("task-io", hasTask ? task.ioPercent + " %" : "—");
  setText("task-priority", hasTask ? fmtInt(task.basePriority) : "—");

  const tp = snapshot.taskParams ?? {};
  setText(
    "task-ranges",
    `Диапазоны генерации: размер ${tp.sizeMin}–${tp.sizeMax} ед. · команд $
{tp.commandsMin}–${tp.commandsMax} · ВВ ${tp.ioPercentMin}–${tp.ioPercentMax} % ·
приоритет ${tp.priorityMin}–${tp.priorityMax}`
  );
}

function renderAverages(snapshot) {
  const stats = snapshot.stats ?? {};

  setText("avg-loaded-count", fmtInt(stats.loadedCount) + " загружено");
  setText("avg-size", fmt1(stats.avgSize));
  setText("avg-commands", fmt1(stats.avgCommands));
  setText("avg-io-percent", fmt1(stats.avgIoPercent) + " %");
  setText("avg-priority", fmt1(stats.avgPriority));

  setText("avg-completed-count", fmtInt(stats.completedCount) + " завершено");
  setText("avg-wait", fmt1(stats.avgWait));
  setText("avg-exec", fmt1(stats.avgExec));
  setText("avg-io", fmt1(stats.avgIo));
  setText("avg-turnaround", fmt1(stats.avgTurnaround));
}

function renderHistory(snapshot) {
  const history = (snapshot.stats ?? {}).execHistory ?? [];
  const box = $("exec-history");

  if (!history.length) {
    box.innerHTML = `<span class="empty-steps">процессы на ЦПр ещё не
выбирались</span>`;
  } else {
    box.innerHTML = history
      .map((id, index) =>
        index === history.length - 1
          ? `<span class="step current">${id}</span>`
          : `<span class="step">${id}</span>`
      )
      .join("");
  }

  setText("history-count", `${history.length} / 16`);
}

function renderReady(snapshot) {
  const rows = snapshot.ready ?? [];
  setText("ready-count", fmtInt(rows.length));
```

```
  renderRows("ready-table", rows, "ready-empty", (p) => `
    <tr>
      <td class="mono">#${p.id}</td>
      <td>${stateMark(p.state)}</td>
      <td class="mono">${p.pc}</td>
      <td class="mono">${p.size}</td>
      <td class="mono">${p.dynamicPriority}</td>
    </tr>`);
}

function renderIO(snapshot) {
  const rows = snapshot.operations ?? [];
  setText("io-count", fmtInt(rows.length));

  renderRows("io-table", rows, "io-empty", (op) => {
    const done = op.totalTicks > 0
      ? Math.round(((op.totalTicks - op.ticksLeft) / op.totalTicks) * 100)
      : 0;
    return `
    <tr>
      <td class="mono">#${op.processorId}</td>
      <td class="mono">#${op.processId}</td>
      <td>${stateMark(op.state)}</td>
      <td class="mono">${op.ticksLeft} / ${op.totalTicks}</td>
      <td class="mono">${done} %</td>
    </tr>`;
  });
}

function renderInterrupts(snapshot) {
  const rows = [...(snapshot.interruptLog ?? [])].reverse();
  setText("int-count", fmtInt(snapshot.interruptsHandled));

  renderRows("int-table", rows, "int-empty", (s) => `
    <tr>
      <td class="mono">${s.time}</td>
      <td>${INTERRUPT_NAMES[s.type] ?? "?"}</td>
      <td class="mono">BB #${s.sourceId}</td>
      <td class="mono">#${s.processId}</td>
    </tr>`);
}

function renderProcesses(snapshot) {
  const rows = snapshot.processes ?? [];
  setText("proc-count", `${rows.length} / ${snapshot.maxProcesses}`);

  renderRows("proc-table", rows, "proc-empty", (p) => `
    <tr class="${p.state === 2 ? "row-active" : ""}">
      <td class="mono">#${p.id}</td>
      <td>${stateMark(p.state)}</td>
      <td class="mono">${p.pc}</td>
      <td class="mono">${p.totalCommands}</td>
      <td class="mono">${p.size}</td>
      <td class="mono">${p.ioPercent} %</td>
      <td class="mono">${p.basePriority} → ${p.dynamicPriority}</td>
      <td class="mono">${p.waitTicks}</td>
      <td class="mono">${p.execTicks}</td>
      <td class="mono">${p.ioTicks}</td>
      <td class="mono">${p.turnaround}</td>
    </tr>`);
}

// ---------------------------------------------------------------------------
// Отрисовка снимка состояния
// ---------------------------------------------------------------------------
```

```
function render(snapshot) {
  renderHeader(snapshot);
  renderSystem(snapshot);
  renderCPU(snapshot);
  renderTask(snapshot);
  renderAverages(snapshot);
  renderHistory(snapshot);
  renderReady(snapshot);
  renderIO(snapshot);
  renderInterrupts(snapshot);
  renderProcesses(snapshot);
}

// ---------------------------------------------------------------------------
// Приём снимков состояния из ядра
// ---------------------------------------------------------------------------

if (window.runtime) {
  window.runtime.EventsOn("os:update", render);
} else {
  console.warn("Wails runtime не найден — запустите приложение через wails dev / wails
build");
}
```

## Результат

В соответствии с индивидуальным заданием (вариант 11 — относительные и динамические приоритеты) получены следующие результаты.

Формула выбора следующего процесса для обслуживания на ЦПр — процесс с наибольшим динамическим приоритетом:

$$P = \text{argmax}(\text{DynamicPriority}),$$

выбор выполняется только при освобождении ЦПр (активный процесс не вытесняется), динамический приоритет ожидающего процесса увеличивается каждый такт:

$$\text{DynamicPriority} = \text{DynamicPriority} + \text{Size}.$$

Показатели эффективности распределения времени, выводимые моделью:

$$\text{полезная загрузка ЦПр} = T_{занят} / T \cdot 100\ \%;$$

$$\text{производительность системы} = N_{выполнено} / T \cdot 1000;$$

$$\text{средняя длина очереди готовности} = \Sigma L / T, \quad \text{максимальная длина} = L_{max};$$

$$\text{среднее время ожидания} = \Sigma \text{WaitTicks} / M, \quad \text{среднее время выполнения} = \Sigma \text{ExecTicks} / M;$$

среднее время ВВ = Σ IOTicks / M,  среднее оборотное время = Σ (t_завер − t_загр) / M,

где T — модельное время в тактах, M — число завершённых процессов.

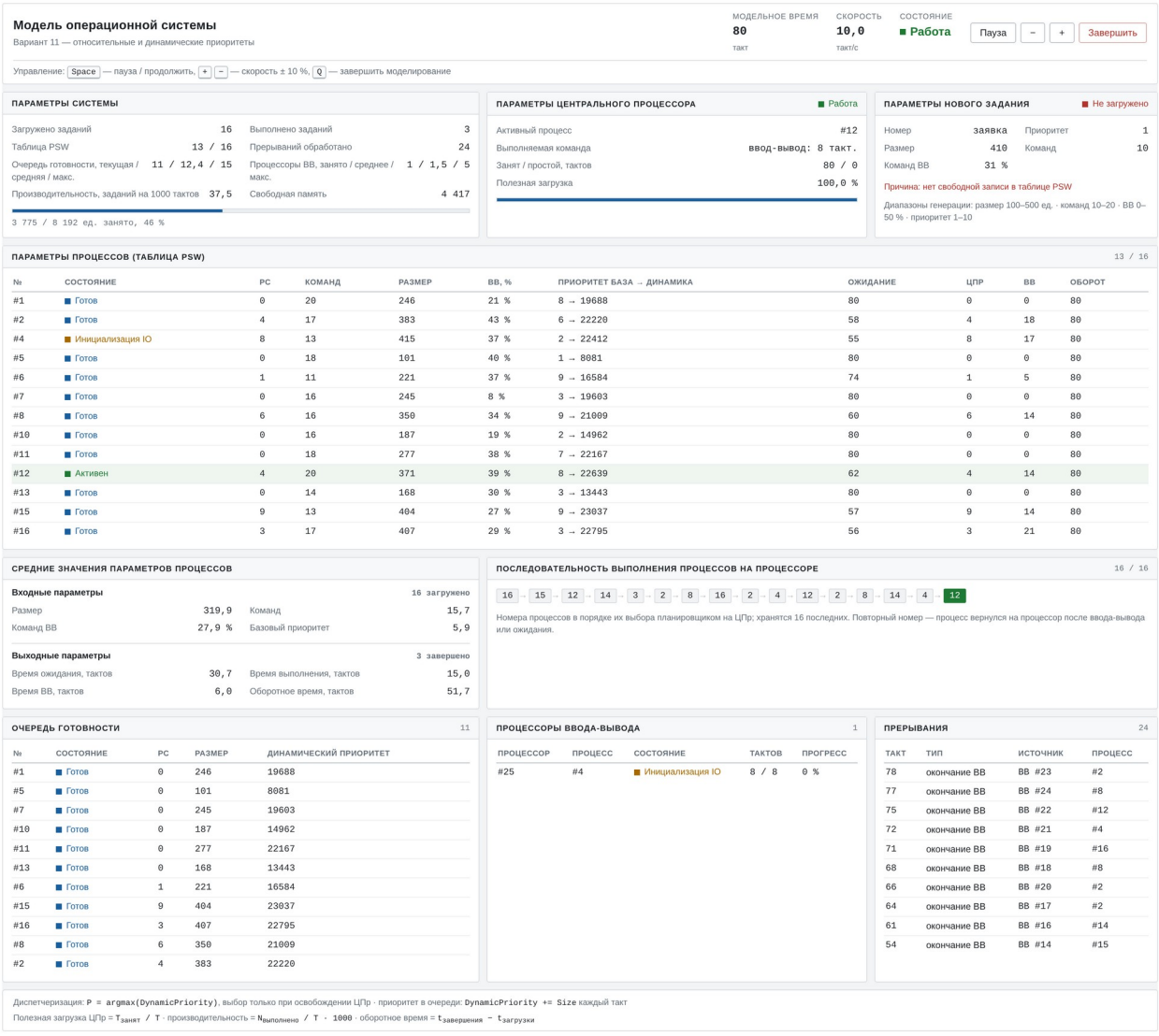



Рисунок 6 — Общий вид модели ОС (все данные на одном экране)

На рисунке 6 приведён общий вид модели: вся информация располагается на одном экране и обновляется без смещения элементов. Показан момент модельного времени 80 такт: загружено 16 заданий, выполнено 3, в таблице PSW находятся 13 процессов, занято 3775 единиц памяти из 8192. Полезная загрузка ЦПр составляет 100 % — в очереди готовности постоянно были процессы; производительность — 37,5 задания

на 1000 тактов. В очереди готовности 11 процессов (средняя длина за прогон 12,4, максимум 15), занят один процессор BB (максимум за прогон — 5).

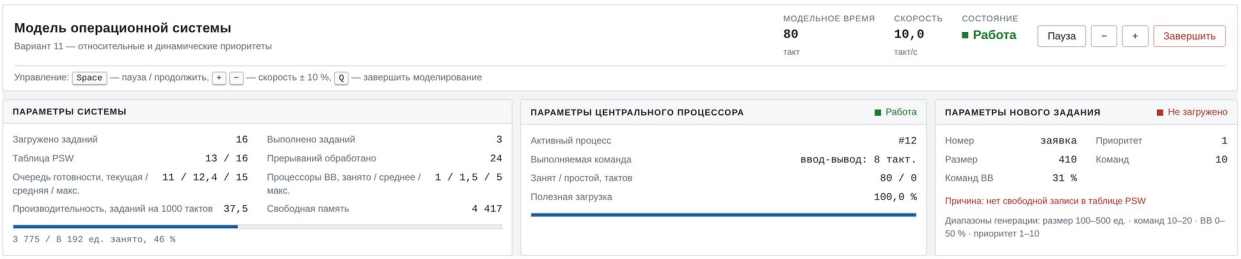



Рисунок 7 — Постоянная информация, параметры системы, ЦПр и нового задания

На рисунке 7 видны постоянная информация (заголовок, состояние модели, язык управления, названия параметров) и параметры системы, центрального процессора и нового задания. Активный процесс #12 выполняет команду ввода-вывода длительностью 8 тактов. Новое задание не загружено: все 16 записей таблицы слов состояний процессов заняты — интерфейс выводит параметры заявки и причину отказа («нет свободной записи в таблице PSW»).

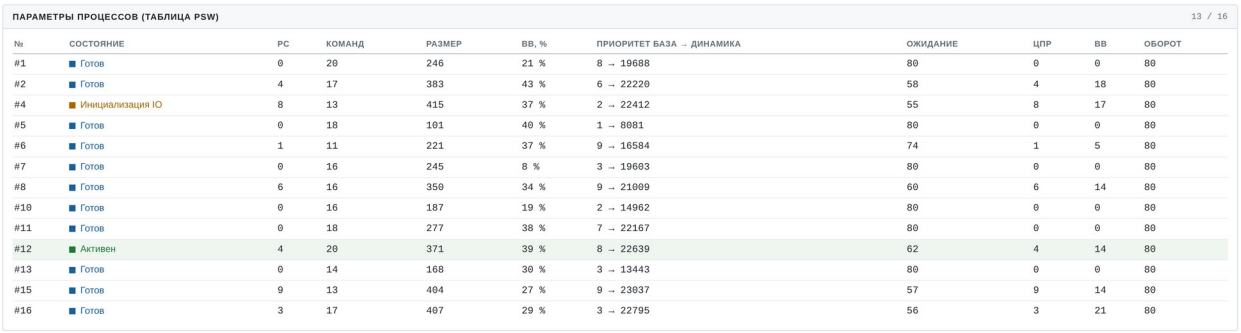
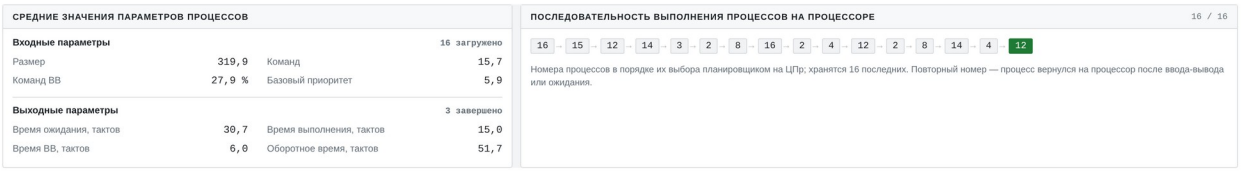



Рисунок 8 — Параметры процессов (таблица PSW) с временами процессов

На рисунке 8 приведена таблица параметров всех процессов: номер, состояние, счётчик команд, число команд, размер, процент команд BB, базовый и динамический приоритеты, а также времена ожидания, выполнения на ЦПр, ввода-вывода и оборотное время. Например, процесс #12 провёл 62 такта в ожидании, 4 такта выполнялся на ЦПр и 14 тактов — в состоянии ввода-вывода; оборотное время — 80 тактов. Процесс #4 в момент съёмки находится в состоянии «Инициализация IO». Видно, что динамические приоритеты ожидающих процессов существенно выросли (например, процесс #5 с размером 101 имеет приоритет 8081, а процесс #15 с

размером 404 — 23037): чем больше размер задания и время ожидания, тем выше приоритет.



СРЕДНИЕ ЗНАЧЕНИЯ ПАРАМЕТРОВ ПРОЦЕССОВ

**Входные параметры** — 16 загружено

| | | | |
|---|---|---|---|
| Размер | 319,9 | Команд | 15,7 |
| Команд BB | 27,9 % | Базовый приоритет | 5,9 |

**Выходные параметры** — 3 завершено

| | | | |
|---|---|---|---|
| Время ожидания, тактов | 30,7 | Время выполнения, тактов | 15,0 |
| Время BB, тактов | 6,0 | Оборотное время, тактов | 51,7 |

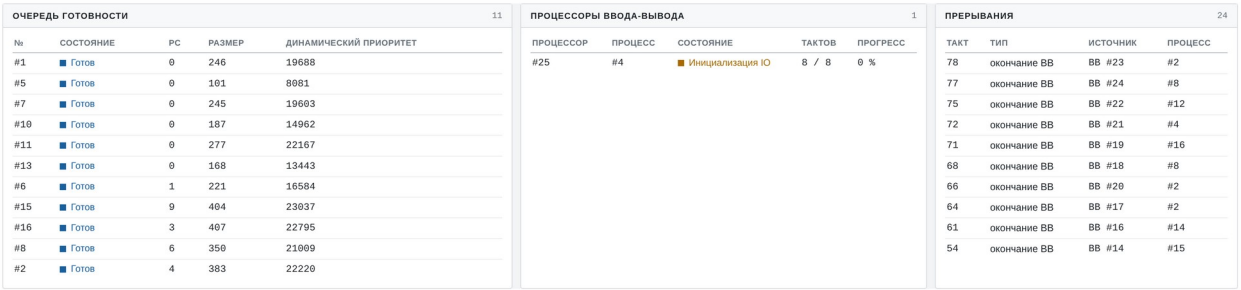
ПОСЛЕДОВАТЕЛЬНОСТЬ ВЫПОЛНЕНИЯ ПРОЦЕССОВ НА ПРОЦЕССОРЕ — 16 / 16

16 → 15 → 12 → 14 → 3 → 2 → 8 → 16 → 2 → 4 → 12 → 2 → 8 → 14 → 4 → 12

Номера процессов в порядке их выбора планировщиком на ЦПр; хранятся 16 последних. Повторный номер — процесс вернулся на процессор после ввода-вывода или ожидания.

Рисунок 9 — Средние значения параметров процессов и последовательность выполнения процессов

На рисунке 9 показаны средние значения входных параметров (по 16 загруженным заданиям: средний размер 319,9 единицы, среднее число команд 15,7, средний процент команд BB 27,9 %, средний базовый приоритет 5,9) и выходных параметров (по 3 завершённым процессам: среднее время ожидания 30,7 такта, среднее время выполнения 15,0 тактов, среднее время BB 6,0 тактов, среднее оборотное время 51,7 такта). Здесь же выводится последовательность выполнения процессов на процессоре — 16 последних номеров: повторяющиеся номера соответствуют возвращению процессов на ЦПр после завершения операций ввода-вывода.



ОЧЕРЕДЬ ГОТОВНОСТИ — 11

| № | СОСТОЯНИЕ | PC | РАЗМЕР | ДИНАМИЧЕСКИЙ ПРИОРИТЕТ |
|---|---|---|---|---|
| #1 | Готов | 0 | 246 | 19688 |
| #5 | Готов | 0 | 101 | 8081 |
| #7 | Готов | 0 | 245 | 19603 |
| #10 | Готов | 0 | 187 | 14962 |
| #11 | Готов | 0 | 277 | 22167 |
| #13 | Готов | 0 | 168 | 13443 |
| #6 | Готов | 1 | 221 | 16584 |
| #15 | Готов | 9 | 404 | 23037 |
| #16 | Готов | 3 | 407 | 22795 |
| #8 | Готов | 6 | 350 | 21009 |
| #2 | Готов | 4 | 383 | 22220 |

ПРОЦЕССОРЫ ВВОДА-ВЫВОДА — 1

| ПРОЦЕССОР | ПРОЦЕСС | СОСТОЯНИЕ | ТАКТОВ | ПРОГРЕСС |
|---|---|---|---|---|
| #25 | #4 | Инициализация IO | 8 / 8 | 0 % |

ПРЕРЫВАНИЯ — 24

| ТАКТ | ТИП | ИСТОЧНИК | ПРОЦЕСС |
|---|---|---|---|
| 78 | окончание BB | BB #23 | #2 |
| 77 | окончание BB | BB #24 | #8 |
| 75 | окончание BB | BB #22 | #12 |
| 72 | окончание BB | BB #21 | #4 |
| 71 | окончание BB | BB #19 | #16 |
| 68 | окончание BB | BB #18 | #8 |
| 66 | окончание BB | BB #20 | #2 |
| 64 | окончание BB | BB #17 | #2 |
| 61 | окончание BB | BB #16 | #14 |
| 54 | окончание BB | BB #14 | #15 |

Рисунок 10 — Очередь готовности, процессоры ввода-вывода и прерывания

На рисунке 10 приведены вспомогательные разделы индикации: очередь готовности с динамическими приоритетами процессов (вариант 11), занятый процессор BB (процессор #25 обслуживает процесс #4) и журнал сигналов прерываний по окончании операций ввода-вывода. Прерывания обрабатывает супервизор прерываний, который через регулировщик возвращает процессы в состояние «Готов».

Таким образом, модель выводит все данные, необходимые для анализа эффективности метода распределения времени ЦПр: времена ожидания и

выполнения, длины очередей, полезную загрузку процессора и производительность системы, а также средние значения входных и выходных параметров процессов. При изменении параметров задания и системы (размер памяти, диапазоны генерации, длительность ВВ) показатели автоматически пересчитываются.

## Вывод

В ходе данной работы разработан интерфейс модели индикации параметров ПО «Модель ОС». Подпрограмма индикации разделена на семь подпрограмм в соответствии с заданием: вывод постоянной информации, параметров системы, параметров нового задания, параметров центрального процессора, параметров процессов, последовательности выполнения процессов на процессоре и средних значений параметров процессов. В модель добавлены необходимые переменные и новый модуль статистики (internal/stats), ведётся учёт времён ожидания, выполнения и ввода-вывода, длин очередей, полезной загрузки процессора, производительности системы и оборотного времени процессов. Предусмотрен вывод средних значений входных и выходных параметров процессов. Вся информация отображается сразу на одном экране; частота обновления ограничена, позиции разделов фиксированы, поэтому изображение стабильно. Корректность учёта статистики подтверждена автоматическими тестами. Получены практические навыки построения подсистемы индикации параметров модели операционной системы.